DropRun - Project Report

Project Files

README.md

```
**Hyper-local order aggregator** — pool your Blinkit/Zepto/Instamart orders with
your building mates through a single Captain who places one consolidated order.

---

## Tech Stack

| Layer | Choice |
|-------|--------|
| Frontend | React 18 + TypeScript |
| Styling | Tailwind CSS v3 |
| Realtime DB | Firebase Firestore (onSnapshot listeners) |
| Auth | Firebase Anonymous Auth |
| Payments | UPI deep-link + QR code (no gateway) |
| Hosting | Firebase Hosting / Vercel |

---

## Setup Instructions

### 1. Clone & Install

```bash
git clone <repo>
cd droprun
npm install
```
```

2. Create a Firebase Project

1. Go to Firebase Console
2. Create a new project
3. Enable **Anonymous Authentication** (Authentication → Sign-in method → Anonymous)
4. Create a **Firestore Database** (start in test mode, then apply the rules below)
5. Go to **Project Settings → General → Your apps** and add a Web app
6. Copy the config object

3. Configure Environment Variables

Edit `.env` in the project root:

```
VITE_FIREBASE_API_KEY=your_api_key
VITE_FIREBASE_AUTH_DOMAIN=your_project.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=your_project_id
```

```
VITE_FIREBASE_STORAGE_BUCKET=your_project.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=your_sender_id
VITE_FIREBASE_APP_ID=your_app_id
VITE_FIREBASE_DATABASE_URL=https://your_project-default-rtdb.firebaseio.com
```

4. Deploy Firestore Security Rules

Paste the contents of `firestore.rules` in the Firebase Console, or:

```
npm install -g firebase-tools
firebase login
firebase init firestore
firebase deploy --only firestore:rules
```

5. Run Locally

```
npm run dev
# Visit http://localhost:5173
```

App Flow

```
Home
├── Create Room (Captain)
│   └── → Captain Dashboard
│       ├── Share 6-digit Room Code
│       ├── Watch live item feed / mark Added / OOS
│       ├── Lock Room → Payment Splits page
│       ├── Mark as Ordered → Waiting banner on all members
│       └── Mark as Delivered → Handoff screen
│
└── Join Room (Member) — enter 6-digit code
    └── → Member Dashboard
        ├── Add items (link, name, price, qty, OOS pref)
        ├── Watch live item status updates
        └── Pay Captain via UPI QR (visible after lock)
```

Firestore Data Model

`rooms/{roomCode}`

```
{
  "roomCode": "ABC123",
  "captainUid": "uid",
  "captainName": "Arjun",
  "captainUPI": "arjun@upi",
  "building": "Hostel Block C",
  "platform": "Blinkit",
```

```
  "status": "open | locked | ordering | delivered | dissolved",
  "expiryTime": 1714000000000,
  "threshold": 200,
  "deliveryFee": 30,
  "totalCartValue": 0,
  "memberCount": 1,
  "createdAt": 1714000000000
}
```

`rooms/{roomCode}/items/{itemId}`

```
{
  "memberUid": "uid",
  "memberName": "Rahul",
  "itemLink": "https://blinkit.com/...",
  "itemName": "Lays Magic Masala 26g",
  "price": 20,
  "quantity": 3,
  "subtotal": 60,
  "oosPreference": "substitute | cancel",
  "status": "pending | added | oos",
  "submittedAt": 1714000000000
}
```

`rooms/{roomCode}/payments/{memberUid}`

```
{ "memberUid": "uid", "paymentStatus": "pending | paid" }
```

Features

- ✅ Anonymous auth — no signup required
- ✅ 6-digit room code generation
- ✅ Real-time updates via Firestore `onSnapshot` across all clients
- ✅ Countdown timer (amber at 3 min, red at 1 min)
- ✅ Auto 5-min extension if ≥75% of threshold hit at expiry
- ✅ Auto-dissolve if threshold not hit
- ✅ Duplicate link detection with confirmation dialog
- ✅ OOS handling: substitute or cancel (auto-adjusts totals)
- ✅ Proportional delivery fee splitting
- ✅ UPI deep-link + QR code (qrcode.react)
- ✅ Captain payment toggle + member self-report
- ✅ Captain re-entry via room code (matched by UID)
- ✅ Late joiner protection
- ✅ Demo session page for presentations

Deployment

Vercel

```
npm run build
# Push to GitHub, connect to Vercel
# Add all VITE_FIREBASE_* env vars in the Vercel dashboard
```

Firebase Hosting

```
firebase init hosting   # public dir: dist, SPA: yes
npm run build
firebase deploy
```

License

MIT — built as a demo project. \n```\n\n### index.html

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <!-- SEO -->
    <title>DropRun — Hyper-local Order Pooling for Your Building</title>
    <meta name="description" content="Pool your Blinkit, Zepto & Swiggy Instamart
orders with your hostel or building mates. One Captain, one delivery, zero
individual fees." />
    <meta name="theme-color" content="#4F46E5" />

    <!-- Open Graph -->
    <meta property="og:title" content="DropRun — Group Order Pooling" />
    <meta property="og:description" content="Order together, save together. Pool
quick-commerce orders with your building." />
    <meta property="og:type" content="website" />

    <!-- Favicon -->
    <link rel="icon" type="image/svg+xml" href="data:image/svg+xml,<svg
xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100 100'><rect width='100'
height='100' rx='20' fill='%234F46E5'/><text y='.9em' font-size='70' x='15'>⚡</text>
</svg>" />

    <!-- Preconnect for Google Fonts (loaded in CSS) -->
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
```

```
</html>
\n```\n\n### schema.sql

```\n-- ============================================================
-- DropRun — Supabase SQL Schema (v2 — public anon access)
-- Run this entire file in your Supabase SQL Editor
-- ==

-- ── TABLES ──

create table if not exists rooms (
  room_code   text primary key,
  captain_uid   text not null,
  captain_name   text not null,
  captain_upi   text not null,
  building   text not null,
  platform   text not null,
  status   text not null default 'open',
  expiry_time   bigint not null,
  threshold   int not null default 200,
  delivery_fee   int not null default 30,
  total_cart_value int not null default 0,
  member_count   int not null default 1,
  created_at   bigint not null
);

create table if not exists items (
  id   uuid primary key default gen_random_uuid(),
  room_code   text not null references rooms(room_code) on delete cascade,
  member_uid   text not null,
  member_name   text not null,
  item_link   text not null,
  item_name   text not null,
  price   int not null,
  quantity   int not null,
  subtotal   int not null,
  oos_preference text not null default 'substitute',
  status   text not null default 'pending',
  submitted_at   bigint not null
);

create table if not exists payments (
  id   uuid primary key default gen_random_uuid(),
  room_code   text not null references rooms(room_code) on delete cascade,
  member_uid   text not null,
  member_name   text not null,
  payment_status text not null default 'pending',
  unique (room_code, member_uid)
);

-- ── REALTIME ──
alter publication supabase_realtime add table rooms;
```
```

```
alter publication supabase_realtime add table items;
alter publication supabase_realtime add table payments;

-- ── ATOMIC RPC FUNCTIONS ──────────────────────────────────

create or replace function increment_member_count(p_room_code text)
returns void language plpgsql security definer as $$
begin
  update rooms set member_count = member_count + 1
  where room_code = p_room_code;
end;
$$;

create or replace function add_to_cart_value(p_room_code text, p_amount int)
returns void language plpgsql security definer as $$
begin
  update rooms set total_cart_value = total_cart_value + p_amount
  where room_code = p_room_code;
end;
$$;

-- ── ROW LEVEL SECURITY — public anon access ──────────────────
-- (no Supabase Auth required — UUID is stored client-side)

alter table rooms    enable row level security;
alter table items    enable row level security;
alter table payments enable row level security;

-- Drop old policies if re-running
drop policy if exists "Public read rooms"    on rooms;
drop policy if exists "Auth create rooms"    on rooms;
drop policy if exists "Auth update rooms"    on rooms;
drop policy if exists "Public read items"    on items;
drop policy if exists "Auth add items"       on items;
drop policy if exists "Auth update items"    on items;
drop policy if exists "Public read payments" on payments;
drop policy if exists "Auth manage payments" on payments;
drop policy if exists "Auth update payments" on payments;

-- Rooms: anyone with the anon key can read/write
create policy "Anon read rooms"   on rooms for select using (true);
create policy "Anon insert rooms" on rooms for insert with check (true);
create policy "Anon update rooms" on rooms for update using (true);

-- Items
create policy "Anon read items"   on items for select using (true);
create policy "Anon insert items" on items for insert with check (true);
create policy "Anon update items" on items for update using (true);

-- Payments
create policy "Anon read payments"   on payments for select using (true);
create policy "Anon insert payments" on payments for insert with check (true);
```

```
create policy "Anon update payments" on payments for update using (true);
\n```\n\n### src/App.css

```\n.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
  border: 2px solid transparent;
  transition: border-color 0.3s;
  margin-bottom: 24px;

  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}

.hero {
  position: relative;

  .base,
  .framework,
  .vite {
    inset-inline: 0;
    margin: 0 auto;
  }

  .base {
    width: 170px;
    position: relative;
    z-index: 0;
  }

  .framework,
  .vite {
    position: absolute;
  }

  .framework {
    z-index: 1;
    top: 34px;
    height: 28px;
    transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
      scale(1.4);
  }

  .vite {
```
```

```
    z-index: 0;
    top: 107px;
    height: 26px;
    width: auto;
    transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
      scale(0.8);
  }
}

#center {
  display: flex;
  flex-direction: column;
  gap: 25px;
  place-content: center;
  place-items: center;
  flex-grow: 1;

  @media (max-width: 1024px) {
    padding: 32px 20px 24px;
    gap: 18px;
  }
}

#next-steps {
  display: flex;
  border-top: 1px solid var(--border);
  text-align: left;

  & > div {
    flex: 1 1 0;
    padding: 32px;
    @media (max-width: 1024px) {
      padding: 24px 20px;
    }
  }

  .icon {
    margin-bottom: 16px;
    width: 22px;
    height: 22px;
  }

  @media (max-width: 1024px) {
    flex-direction: column;
    text-align: center;
  }
}

#docs {
  border-right: 1px solid var(--border);

  @media (max-width: 1024px) {
```

```
    border-right: none;
    border-bottom: 1px solid var(--border);
  }
}

#next-steps ul {
  list-style: none;
  padding: 0;
  display: flex;
  gap: 8px;
  margin: 32px 0 0;

  .logo {
    height: 18px;
  }

  a {
    color: var(--text-h);
    font-size: 16px;
    border-radius: 6px;
    background: var(--social-bg);
    display: flex;
    padding: 6px 12px;
    align-items: center;
    gap: 8px;
    text-decoration: none;
    transition: box-shadow 0.3s;

    &:hover {
      box-shadow: var(--shadow);
    }
    .button-icon {
      height: 18px;
      width: 18px;
    }
  }

  @media (max-width: 1024px) {
    margin-top: 20px;
    flex-wrap: wrap;
    justify-content: center;

    li {
      flex: 1 1 calc(50% - 8px);
    }

    a {
      width: 100%;
      justify-content: center;
      box-sizing: border-box;
    }
  }
```

```
}

#spacer {
  height: 88px;
  border-top: 1px solid var(--border);
  @media (max-width: 1024px) {
    height: 48px;
  }
}

.ticks {
  position: relative;
  width: 100%;

  &::before,
  &::after {
    content: '';
    position: absolute;
    top: -4.5px;
    border: 5px solid transparent;
  }

  &::before {
    left: 0;
    border-left-color: var(--border);
  }
  &::after {
    right: 0;
    border-right-color: var(--border);
  }
}
\n```\n\n### src/App.tsx

```\n
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { Toaster } from 'react-hot-toast';
import { AuthProvider } from './AuthContext';
import { HomePage } from './pages/HomePage';
import { CreateRoomPage } from './pages/CreateRoomPage';
import { JoinRoomPage } from './pages/JoinRoomPage';
import { CaptainDashboard } from './pages/CaptainDashboard';
import { MemberDashboard } from './pages/MemberDashboard';
import { PaymentSplitsPage } from './pages/PaymentSplitsPage';
import { DemoPage } from './pages/DemoPage';

function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/" element={<HomePage />} />
          <Route path="/create" element={<CreateRoomPage />} />
```
```

```
          <Route path="/join" element={<JoinRoomPage />} />
          <Route path="/captain/:roomCode" element={<CaptainDashboard />} />
          <Route path="/member/:roomCode" element={<MemberDashboard />} />
          <Route path="/splits/:roomCode" element={<PaymentSplitsPage />} />
          <Route path="/demo" element={<DemoPage />} />
          <Route path="*" element={<Navigate to="/" replace />} />
        </Routes>
      </BrowserRouter>

      <Toaster
        position="top-right"
        toastOptions={{
          duration: 4000,
          style: {
            background: 'rgba(17, 17, 32, 0.95)',
            color: '#fff',
            border: '1px solid rgba(255,255,255,0.1)',
            backdropFilter: 'blur(20px)',
            borderRadius: '12px',
            fontSize: '14px',
            fontFamily: 'Inter, sans-serif',
            boxShadow: '0 8px 32px rgba(0,0,0,0.4)',
          },
          success: {
            iconTheme: { primary: '#10B981', secondary: '#fff' },
          },
          error: {
            iconTheme: { primary: '#EF4444', secondary: '#fff' },
          },
        }}
      />
    </AuthProvider>
  );
}

export default App;
\n```\n\n### src/AuthContext.tsx

```\nimport React, { createContext, useContext, useState } from 'react';

// ── Stable per-browser user ID stored in localStorage ─────────
function getOrCreateUserId(): string {
  const key = 'droprun_uid';
  let uid = localStorage.getItem(key);
  if (!uid) {
    // crypto.randomUUID() is available in all modern browsers
    uid = crypto.randomUUID();
    localStorage.setItem(key, uid);
  }
  return uid;
}
```
```

```
interface AuthContextType {
  userId: string;
  displayName: string;
  setDisplayName: (name: string) => void;
}

const AuthContext = createContext<AuthContextType>({
  userId: '',
  displayName: '',
  setDisplayName: () => {},
});

export const AuthProvider: React.FC<{ children: React.ReactNode }> = ({ children })
=> {
  // userId is stable across page reloads
  const [userId] = useState<string>(getOrCreateUserId);
  const [displayName, setDN] = useState<string>(
    () => localStorage.getItem('droprun_displayName') || ''
  );

  const setDisplayName = (name: string) => {
    localStorage.setItem('droprun_displayName', name);
    setDN(name);
  };

  return (
    <AuthContext.Provider value={{ userId, displayName, setDisplayName }}>
      {children}
    </AuthContext.Provider>
  );
};

// Convenience hook — exposes { userId, displayName, setDisplayName }
// Also exposes a `user` shim so existing pages that use `user?.id` still work
export const useAuth = () => {
  const ctx = useContext(AuthContext);
  return {
    ...ctx,
    // shim: pages do `user?.id` — keep working without any changes
    user: { id: ctx.userId },
    loading: false,
  };
};
\n```\n\n### src/components/CountdownTimer.tsx

```\nimport React, { useEffect, useState } from 'react';
import { formatCountdown, getCountdownClass } from '../utils';

interface CountdownTimerProps {
  expiryTime: number;
  onExpired?: () => void;
  large?: boolean;
```
```

```
}

export const CountdownTimer: React.FC<CountdownTimerProps> = ({ expiryTime,
onExpired, large }) => {
  const [remaining, setRemaining] = useState(expiryTime - Date.now());

  useEffect(() => {
    const interval = setInterval(() => {
      const r = expiryTime - Date.now();
      setRemaining(r);
      if (r <= 0 && onExpired) {
        onExpired();
      }
    }, 1000);
    return () => clearInterval(interval);
  }, [expiryTime, onExpired]);

  const display = formatCountdown(Math.max(0, remaining));
  const cls = getCountdownClass(remaining);

  if (large) {
    return (
      <div className={`font-mono font-bold text-4xl tracking-widest ${cls || 'text-
white'}`}>
        {display}
      </div>
    );
  }
  return (
    <span className={`font-mono font-semibold text-lg ${cls || 'text-white/80'}`}>
      {display}
    </span>
  );
};
\n```\n\n### src/components/Loading.tsx

```\nimport React from 'react';
import { Loader2 } from 'lucide-react';

export const LoadingScreen: React.FC<{ text?: string }> = ({ text = 'Loading...' })
=> (
  <div className="min-h-screen flex flex-col items-center justify-center gap-4">
    <div className="relative">
      <div className="w-16 h-16 rounded-full border-2 border-indigo-500/30 border-t-
indigo-500 animate-spin" />
      <div className="absolute inset-0 flex items-center justify-center">
        <div className="w-8 h-8 rounded-full bg-indigo-600/20 flex items-center
justify-center">
          <Loader2 className="w-4 h-4 text-indigo-400 animate-spin" />
        </div>
      </div>
    </div>
```
```

```
    <p className="text-white/50 text-sm">{text}</p>
  </div>
);

export const SkeletonCard: React.FC = () => (
  <div className="card p-4 space-y-3 animate-pulse">
    <div className="flex gap-3 items-center">
      <div className="skeleton w-8 h-8 rounded-full" />
      <div className="space-y-1 flex-1">
        <div className="skeleton h-4 w-1/3 rounded" />
        <div className="skeleton h-3 w-1/2 rounded" />
      </div>
      <div className="skeleton h-6 w-16 rounded-full" />
    </div>
    <div className="skeleton h-3 w-full rounded" />
    <div className="skeleton h-3 w-3/4 rounded" />
  </div>
);

interface EmptyStateProps {
  icon: React.ReactNode;
  title: string;
  description: string;
  action?: React.ReactNode;
}

export const EmptyState: React.FC<EmptyStateProps> = ({ icon, title, description,
action }) => (
  <div className="flex flex-col items-center justify-center py-16 px-8 text-center
space-y-4">
    <div className="w-16 h-16 rounded-2xl bg-white/5 border border-white/10 flex
items-center justify-center text-white/30 text-3xl">
      {icon}
    </div>
    <div className="space-y-1">
      <p className="text-white/70 font-semibold">{title}</p>
      <p className="text-white/40 text-sm max-w-xs">{description}</p>
    </div>
    {action}
  </div>
);
\n```\n\n### src/components/ProgressBar.tsx

```\nimport React, { useEffect, useRef } from 'react';

interface ProgressBarProps {
  value: number;   // current value
  threshold: number; // target
  animate?: boolean; // pulse on update
}

export const ProgressBar: React.FC<ProgressBarProps> = ({ value, threshold, animate
```
```

```
}) => {
  const pct = Math.min((value / threshold) * 100, 100);
  const reached = pct >= 100;
  const barRef = useRef<HTMLDivElement>(null);
  const prevPct = useRef(pct);

  useEffect(() => {
    if (pct !== prevPct.current && barRef.current) {
      barRef.current.classList.add('threshold-reached');
      setTimeout(() => barRef.current?.classList.remove('threshold-reached'), 3000);
    }
    prevPct.current = pct;
  }, [pct]);

  return (
    <div className="space-y-2">
      <div className="flex justify-between text-sm">
        <span className="text-white/60">Cart Value</span>
        <span className={reached ? 'text-emerald-400 font-bold' : 'text-white/80'}>
          ₹{value.toFixed(0)}{' '}
          <span className="text-white/40">/ ₹{threshold}</span>
          {reached && <span className="ml-1 text-emerald-400">✓ Free delivery!
</span>}
        </span>
      </div>
      <div className="progress-bar-container" ref={barRef}>
        <div
          className={`progress-bar-fill ${animate ? 'animate-bounce-subtle' : ''}`}
          style={{ width: `${pct}%` }}
        />
        {!reached && (
          <div className="absolute inset-0 flex items-center justify-end pr-2">
            <span className="text-[10px] text-white/40 font-medium">
              ₹{Math.max(0, threshold - value).toFixed(0)} to go
            </span>
          </div>
        )}
      </div>
    </div>
  );
};
\n```\n\n### src/components/StatusBadge.tsx

```\nimport React from 'react';
import type { RoomStatus } from '../types';

interface StatusBadgeProps {
  status: RoomStatus;
}

const labels: Record<RoomStatus, string> = {
  open: 'OPEN',
```
```

```
  locked: 'LOCKED',
  ordering: 'ORDERING',
  delivered: 'DELIVERED',
  dissolved: 'DISSOLVED',
};

const classes: Record<RoomStatus, string> = {
  open: 'badge-open',
  locked: 'badge-locked',
  ordering: 'badge-ordering',
  delivered: 'badge-delivered',
  dissolved: 'badge-dissolved',
};

const dots: Record<RoomStatus, string> = {
  open: 'bg-emerald-400',
  locked: 'bg-amber-400',
  ordering: 'bg-indigo-400',
  delivered: 'bg-purple-400',
  dissolved: 'bg-red-400',
};

export const StatusBadge: React.FC<StatusBadgeProps> = ({ status }) => (
  <span className={classes[status]}>
    <span className={`w-1.5 h-1.5 rounded-full mr-1.5 ${dots[status]} ${status ===
'open' ? 'animate-pulse' : ''}`} />
    {labels[status]}
  </span>
);
\n```\n\n### src/components/UPIModal.tsx

```\nimport React, { useState } from 'react';
import { X, Loader2 } from 'lucide-react';
import { QRCodeCanvas } from 'qrcode.react';
import { generateUPILink } from '../utils';
import toast from 'react-hot-toast';
import { supabase } from '../supabase';

interface UPIModalProps {
  amount: number;
  captainUPI: string;
  captainName: string;
  roomCode: string;
  memberUid: string;
  memberName: string;
  onClose: () => void;
  onPaid: () => void;
}

export const UPIModal: React.FC<UPIModalProps> = ({
  amount, captainUPI, captainName, roomCode, memberUid, memberName, onClose, onPaid,
}) => {
```
```

```
  const [marking, setMarking] = useState(false);
  const upiLink = generateUPILink({ upiId: captainUPI, amount, name: captainName,
roomCode });

  const handleMarkPaid = async () => {
    setMarking(true);
    try {
      const { error } = await supabase.from('payments').upsert(
        { room_code: roomCode, member_uid: memberUid, member_name: memberName,
payment_status: 'paid' },
        { onConflict: 'room_code,member_uid' }
      );
      if (error) throw error;
      toast.success('Marked as paid!');
      onPaid();
      onClose();
    } catch (err) {
      console.error(err);
      toast.error('Failed to update payment status');
    } finally {
      setMarking(false);
    }
  };

  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-
black/70 backdrop-blur-sm">
      <div className="card w-full max-w-sm p-6 space-y-5 relative">
        <button onClick={onClose} className="absolute top-4 right-4 text-white/30
hover:text-white transition-colors">
          <X className="w-5 h-5" />
        </button>

        <div className="text-center space-y-1">
          <h2 className="text-white font-bold text-lg">Pay {captainName}</h2>
          <p className="text-white/40 text-sm">via UPI</p>
        </div>

        <div className="flex justify-center">
          <div className="bg-white p-3 rounded-xl">
            <QRCodeCanvas value={upiLink} size={180} />
          </div>
        </div>

        <div className="text-center space-y-0.5">
          <div className="text-3xl font-black text-white">₹{amount}</div>
          <div className="text-white/40 text-sm font-mono">{captainUPI}</div>
        </div>

        <a
          href={upiLink}
          className="btn-primary flex items-center justify-center gap-2 w-full py-3"
```

```
        >
          Open UPI App
        </a>

        <button
          id="mark-paid-btn"
          onClick={handleMarkPaid}
          disabled={marking}
          className="btn-secondary w-full flex items-center justify-center gap-2 py-
3"
        >
          {marking ? <Loader2 className="w-4 h-4 animate-spin" /> : '✓ Mark as
Paid'}
        </button>

        <p className="text-center text-white/30 text-xs">
          Scan the QR or tap "Open UPI App", then mark as paid once done
        </p>
      </div>
    </div>
  );
};
\n```\n\n### src/firebase.ts

```\n// firebase.ts is no longer used — backend migrated to Supabase
// This file is kept as an empty stub to avoid TS errors during transition
export {};
\n```\n\n### src/index.css

```\n@import url('https://fonts.googleapis.com/css2?
family=Inter:wght@300;400;500;600;700;800;900&display=swap');

@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  * {
    box-sizing: border-box;
  }
  body {
    @apply bg-gray-950 text-white font-sans antialiased;
    background: linear-gradient(135deg, #0f0c29 0%, #1a1a2e 50%, #16213e 100%);
    min-height: 100vh;
  }
}

@layer components {
  .btn-primary {
    @apply bg-indigo-600 hover:bg-indigo-500 active:bg-indigo-700 text-white font-
semibold py-3 px-6 rounded-xl transition-all duration-200 shadow-lg shadow-indigo-
900/30 hover:shadow-indigo-700/40 hover:-translate-y-0.5 active:translate-y-0
```

```
disabled:opacity-50 disabled:cursor-not-allowed disabled:hover:translate-y-0;
  }
  .btn-secondary {
    @apply bg-white/10 hover:bg-white/20 active:bg-white/5 text-white font-semibold
py-3 px-6 rounded-xl transition-all duration-200 border border-white/20
hover:border-white/40 hover:-translate-y-0.5 disabled:opacity-50 disabled:cursor-
not-allowed;
  }
  .btn-success {
    @apply bg-emerald-600 hover:bg-emerald-500 active:bg-emerald-700 text-white
font-semibold py-3 px-6 rounded-xl transition-all duration-200 shadow-lg shadow-
emerald-900/30 hover:-translate-y-0.5 disabled:opacity-50 disabled:cursor-not-
allowed;
  }
  .btn-danger {
    @apply bg-red-600 hover:bg-red-500 text-white font-semibold py-2 px-4 rounded-lg
transition-all duration-200 hover:-translate-y-0.5 disabled:opacity-50
disabled:cursor-not-allowed;
  }
  .btn-warning {
    @apply bg-amber-500 hover:bg-amber-400 text-white font-semibold py-2 px-4
rounded-lg transition-all duration-200 hover:-translate-y-0.5 disabled:opacity-50
disabled:cursor-not-allowed;
  }
  .card {
    @apply bg-white/5 backdrop-blur-lg border border-white/10 rounded-2xl shadow-xl;
  }
  .card-hover {
    @apply card hover:bg-white/10 transition-all duration-200 hover:-translate-y-0.5
hover:border-white/20;
  }
  .input-field {
    @apply bg-white/10 border border-white/20 rounded-xl px-4 py-3 text-white
placeholder-white/40 focus:outline-none focus:border-indigo-500 focus:ring-2
focus:ring-indigo-500/30 transition-all duration-200 w-full;
  }
  .badge {
    @apply inline-flex items-center px-2.5 py-1 rounded-full text-xs font-semibold;
  }
  .badge-open {
    @apply badge bg-emerald-500/20 text-emerald-400 border border-emerald-500/30;
  }
  .badge-locked {
    @apply badge bg-amber-500/20 text-amber-400 border border-amber-500/30;
  }
  .badge-ordering {
    @apply badge bg-indigo-500/20 text-indigo-400 border border-indigo-500/30;
  }
  .badge-delivered {
    @apply badge bg-purple-500/20 text-purple-400 border border-purple-500/30;
  }
  .badge-dissolved {
```

```
    @apply badge bg-red-500/20 text-red-400 border border-red-500/30;
  }
  .chip-pending {
    @apply badge bg-gray-500/20 text-gray-400 border border-gray-500/30;
  }
  .chip-added {
    @apply badge bg-emerald-500/20 text-emerald-400 border border-emerald-500/30;
  }
  .chip-oos {
    @apply badge bg-red-500/20 text-red-400 border border-red-500/30;
  }
  .chip-paid {
    @apply badge bg-emerald-500/20 text-emerald-400 border border-emerald-500/30;
  }
  .chip-unpaid {
    @apply badge bg-amber-500/20 text-amber-400 border border-amber-500/30;
  }
  .progress-bar-container {
    @apply bg-white/10 rounded-full overflow-hidden h-3 relative;
  }
  .progress-bar-fill {
    @apply h-full rounded-full transition-all duration-700 ease-out relative
overflow-hidden;
    background: linear-gradient(90deg, #059669 0%, #10B981 60%, #34d399 100%);
  }
  .progress-bar-fill::after {
    content: '';
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background: linear-gradient(90deg, transparent 0%, rgba(255,255,255,0.3) 50%,
transparent 100%);
    animation: shimmer 2s infinite;
  }
  .skeleton {
    @apply bg-white/10 rounded-lg animate-pulse;
  }
  .glass-panel {
    background: rgba(255, 255, 255, 0.03);
    backdrop-filter: blur(20px);
    border: 1px solid rgba(255, 255, 255, 0.08);
  }
}

@keyframes shimmer {
  0% { transform: translateX(-100%); }
  100% { transform: translateX(100%); }
}

@keyframes threshold-pulse {
```

```
  0%, 100% { box-shadow: 0 0 0 0 rgba(16, 185, 129, 0.4); }
  50% { box-shadow: 0 0 0 10px rgba(16, 185, 129, 0); }
}

.threshold-reached {
  animation: threshold-pulse 1.5s ease-in-out 3;
}

@keyframes slideUpModal {
  from { transform: translateY(30px); opacity: 0; }
  to { transform: translateY(0); opacity: 1; }
}

.modal-enter {
  animation: slideUpModal 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
}

.countdown-amber {
  color: #F59E0B;
  text-shadow: 0 0 20px rgba(245, 158, 11, 0.5);
}

.countdown-red {
  color: #EF4444;
  text-shadow: 0 0 20px rgba(239, 68, 68, 0.5);
  animation: pulse 1s ease-in-out infinite;
}

.item-card-enter {
  animation: slideUpCard 0.3s ease-out;
}

@keyframes slideUpCard {
  from { transform: translateY(20px); opacity: 0; }
  to { transform: translateY(0); opacity: 1; }
}

/* Scrollbar */
::-webkit-scrollbar { width: 6px; height: 6px; }
::-webkit-scrollbar-track { background: rgba(255,255,255,0.05); }
::-webkit-scrollbar-thumb { background: rgba(255,255,255,0.2); border-radius: 3px; }
::-webkit-scrollbar-thumb:hover { background: rgba(255,255,255,0.35); }

/* Selection */
::selection { background: rgba(79, 70, 229, 0.4); color: white; }
\n```\n\n### src/main.tsx

```\nimport { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'
```
```

```
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
\n```\n\n### src/pages/CaptainDashboard.tsx

```\nimport React, { useEffect, useState, useCallback } from 'react';
import { useParams, useNavigate, Link } from 'react-router-dom';
import {
  ArrowLeft, Users, Package, Lock, ShoppingBag, CheckCircle,
  Copy, Share2, ExternalLink, X, Check, Plus, Loader2, Trash2,
} from 'lucide-react';
import { supabase } from '../supabase';
import { useAuth } from '../AuthContext';
import type { Room, ItemSubmission } from '../types';
import type { Platform, RoomStatus, OOSPreference, ItemStatus } from '../types';
import { CountdownTimer } from '../components/CountdownTimer';
import { ProgressBar } from '../components/ProgressBar';
import { StatusBadge } from '../components/StatusBadge';
import { SkeletonCard, EmptyState } from '../components/Loading';
import toast from 'react-hot-toast';

// ── Save visited room to history ──────────────────────────────
function saveRoomToHistory(roomCode: string, building: string, platform: string,
role: 'captain' | 'member') {
  const key = 'droprun_room_history';
  const existing: Array<{ roomCode: string; building: string; platform: string;
role: string; visitedAt: number }> =
    JSON.parse(localStorage.getItem(key) || '[]');
  const filtered = existing.filter(r => r.roomCode !== roomCode);
  filtered.unshift({ roomCode, building, platform, role, visitedAt: Date.now() });
  localStorage.setItem(key, JSON.stringify(filtered.slice(0, 10)));
}

// ── Inline Add-Item Form (reused from MemberDashboard) ──────

interface AddItemFormProps {
  onClose: () => void;
  onSubmit: (data: { itemLink: string; itemName: string; price: number; quantity:
number; subtotal: number; oosPreference: OOSPreference }) => Promise<unknown>;
  allItems: ItemSubmission[];
}

const AddItemForm: React.FC<AddItemFormProps> = ({ onClose, onSubmit, allItems }) =>
{
  const [link, setLink]   = React.useState('');
  const [name, setName]   = React.useState('');
  const [price, setPrice]   = React.useState('');
  const [qty, setQty]   = React.useState('1');
  const [oosPreference, setOos] = React.useState<OOSPreference>('substitute');
  const [saving, setSaving]   = React.useState(false);
```
```

```
  const handleSubmit = async () => {
    if (!link.trim() || !name.trim() || !price) { toast.error('Fill in all fields');
return; }
    const p = parseFloat(price);
    const q = parseInt(qty, 10) || 1;
    if (isNaN(p) || p <= 0) { toast.error('Enter a valid price'); return; }
    const dup = allItems.find(i => i.itemLink === link.trim());
    if (dup) { toast.error(`Already added by ${dup.memberName}`); return; }
    setSaving(true);
    try { await onSubmit({ itemLink: link.trim(), itemName: name.trim(), price: p,
quantity: q, subtotal: p * q, oosPreference }); }
    finally { setSaving(false); }
  };

  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-
black/70 backdrop-blur-sm">
      <div className="card w-full max-w-md p-6 space-y-4 relative">
        <button onClick={onClose} className="absolute top-4 right-4 text-white/30
hover:text-white transition-colors"><X className="w-5 h-5" /></button>
        <h2 className="text-white font-bold text-lg">Add My Items</h2>
        <div className="space-y-3">
          <input id="cap-item-link" className="input-field" placeholder="Item URL
(Blinkit / Zepto / Instamart)" value={link} onChange={e => setLink(e.target.value)}
/>
          <input id="cap-item-name" className="input-field" placeholder="Item name"
value={name} onChange={e => setName(e.target.value)} />
          <div className="grid grid-cols-2 gap-3">
            <div className="relative">
              <span className="absolute left-3.5 top-1/2 -translate-y-1/2 text-
white/30">₹</span>
              <input id="cap-item-price" type="number" className="input-field pl-7"
placeholder="Price" value={price} onChange={e => setPrice(e.target.value)} min="0"
/>
            </div>
            <input id="cap-item-qty" type="number" className="input-field"
placeholder="Qty" value={qty} onChange={e => setQty(e.target.value)} min="1" />
          </div>
          {price && qty && (
            <div className="text-right text-white/60 text-sm">Subtotal: <span
className="text-white font-bold">₹{(parseFloat(price||'0') *
(parseInt(qty,10)||1)).toFixed(0)}</span></div>
          )}
          <div>
            <p className="text-white/50 text-xs mb-2">If out of stock:</p>
            <div className="grid grid-cols-2 gap-2">
              {(['substitute','cancel'] as OOSPreference[]).map(opt => (
                <button key={opt} onClick={() => setOos(opt)} className={`py-2
rounded-xl border text-sm font-semibold transition-all ${ oosPreference === opt ?
'bg-indigo-600/20 border-indigo-500 text-white' : 'border-white/10 text-white/40
hover:border-white/20' }`}>
```

```
                  {opt === 'substitute' ? '↔ Substitute' : '× Cancel'}
                </button>
              ))}
            </div>
          </div>
        </div>
        <div className="grid grid-cols-2 gap-3 pt-1">
          <button onClick={onClose} className="btn-secondary py-2.5">Cancel</button>
          <button id="cap-submit-item-btn" onClick={handleSubmit} disabled={saving}
className="btn-primary flex items-center justify-center gap-2 py-2.5">
            {saving ? <Loader2 className="w-4 h-4 animate-spin" /> : 'Add Item'}
          </button>
        </div>
      </div>
    </div>
  );
};

// ── DB row mappers ──────────────────────────────────────────

const mapRoom = (r: Record<string, unknown>): Room => ({
  roomCode:       r.room_code as string,
  captainUid:     r.captain_uid as string,
  captainName:    r.captain_name as string,
  captainUPI:     r.captain_upi as string,
  building:       r.building as string,
  platform:       r.platform as Platform,
  status:         r.status as RoomStatus,
  expiryTime:     r.expiry_time as number,
  threshold:      r.threshold as number,
  deliveryFee:    r.delivery_fee as number,
  totalCartValue: r.total_cart_value as number,
  memberCount:    r.member_count as number,
  createdAt:      r.created_at as number,
});

const mapItem = (r: Record<string, unknown>): ItemSubmission => ({
  id:            r.id as string,
  memberUid:     r.member_uid as string,
  memberName:    r.member_name as string,
  itemLink:      r.item_link as string,
  itemName:      r.item_name as string,
  price:         r.price as number,
  quantity:      r.quantity as number,
  subtotal:      r.subtotal as number,
  oosPreference: r.oos_preference as OOSPreference,
  status:        r.status as ItemStatus,
  submittedAt:   r.submitted_at as number,
});

// ── Component ───────────────────────────────────────────────
```

```
export const CaptainDashboard: React.FC = () => {
  const { roomCode } = useParams<{ roomCode: string }>();
  const navigate = useNavigate();
  const { user, displayName } = useAuth();
  const [showForm, setShowForm] = useState(false);

  const [room, setRoom]       = useState<Room | null>(null);
  const [items, setItems]     = useState<ItemSubmission[]>([]);
  const [loading, setLoading] = useState(true);
  const [copied, setCopied]   = useState(false);

  // ── fetch helpers ──────────────────────────────────────

  const fetchItems = useCallback(async () => {
    if (!roomCode) return;
    const { data } = await supabase
      .from('items')
      .select('*')
      .eq('room_code', roomCode)
      .order('submitted_at', { ascending: true });
    if (data) setItems(data.map(mapItem));
  }, [roomCode]);

  // ── expiry check ───────────────────────────────────────

  const handleExpiryCheck = useCallback(async (r: Room) => {
    if (r.status !== 'open' || Date.now() < r.expiryTime) return;
    const hit = r.totalCartValue >= r.threshold * 0.75;
    if (hit) {
      const newExpiry = Date.now() + 5 * 60 * 1000;
      await supabase.from('rooms').update({ expiry_time: newExpiry
}).eq('room_code', r.roomCode);
      toast('⏰ Room auto-extended by 5 min — threshold nearly hit!');
    } else {
      await supabase.from('rooms').update({ status: 'dissolved' }).eq('room_code',
r.roomCode);
      toast.error('Room dissolved — threshold not met');
      navigate('/');
    }
  }, [navigate]);

  // ── realtime subscription ──────────────────────────────

  useEffect(() => {
    if (!roomCode) return;
    let cancelled = false;

    const load = async () => {
      const { data: roomRow, error } = await supabase
        .from('rooms')
        .select('*')
        .eq('room_code', roomCode)
```

```
        .single();

      if (error || !roomRow) { navigate('/'); return; }

      const roomData = mapRoom(roomRow);
      if (!cancelled) { setRoom(roomData); setLoading(false); }
      handleExpiryCheck(roomData);

      // Save to history
      saveRoomToHistory(roomCode, roomData.building, roomData.platform, 'captain');

      await fetchItems();
    };

    load();

    const channel = supabase
      .channel(`captain-${roomCode}`)
      .on('postgres_changes', {
        event: '*', schema: 'public', table: 'rooms',
        filter: `room_code=eq.${roomCode}`,
      }, (payload) => {
        if (payload.new) {
          const r = mapRoom(payload.new as Record<string, unknown>);
          setRoom(r);
          handleExpiryCheck(r);
        }
      })
      .on('postgres_changes', {
        event: '*', schema: 'public', table: 'items',
        filter: `room_code=eq.${roomCode}`,
      }, () => fetchItems())
      .subscribe();

    return () => {
      cancelled = true;
      supabase.removeChannel(channel);
    };
  }, [roomCode, navigate, fetchItems, handleExpiryCheck]);

  // ── add own items (captain) ──────────────────────────────

  const handleAddItem = async (data: { itemLink: string; itemName: string; price:
number; quantity: number; subtotal: number; oosPreference: OOSPreference }) => {
    if (!user || !roomCode) return;
    const { error } = await supabase.from('items').insert({
      room_code:      roomCode,
      member_uid:     user.id,
      member_name:    displayName || 'Captain',
      item_link:      data.itemLink,
      item_name:      data.itemName,
      price:          data.price,
```

```
      quantity:       data.quantity,
      subtotal:       data.subtotal,
      oos_preference: data.oosPreference,
      status:         'pending',
      submitted_at:   Date.now(),
    });
    if (error) { toast.error('Failed to add item'); return; }
    await supabase.rpc('add_to_cart_value', { p_room_code: roomCode, p_amount:
data.subtotal });
    toast.success(`"${data.itemName}" added!`);
    setShowForm(false);
  };

  // ── actions ──────────────────────────────────────────────

  const updateStatus = async (status: RoomStatus) => {
    if (!roomCode) return;
    await supabase.from('rooms').update({ status }).eq('room_code', roomCode);
    toast.success(`Room status → ${status}`);
    if (status === 'locked') navigate(`/splits/${roomCode}`);
    if (status === 'dissolved') navigate('/');
  };

  const toggleItemStatus = async (item: ItemSubmission) => {
    const next: ItemStatus =
      item.status === 'pending' ? 'added' :
      item.status === 'added'   ? 'oos'   : 'pending';
    await supabase.from('items').update({ status: next }).eq('id', item.id);
  };

  const copyCode = () => {
    if (!roomCode) return;
    navigator.clipboard.writeText(roomCode);
    setCopied(true);
    toast.success('Room code copied!');
    setTimeout(() => setCopied(false), 2000);
  };

  const shareRoom = () => {
    if (!roomCode) return;
    const url = `${window.location.origin}/join`;
    if (navigator.share) {
      navigator.share({ title: 'Join my DropRun!', text: `Room code: ${roomCode}`,
url });
    } else {
      navigator.clipboard.writeText(url);
      toast.success('Join link copied!');
    }
  };

  // ── render ───────────────────────────────────────────────
```

```
  if (loading) return (
    <div className="min-h-screen flex flex-col">
      <div className="p-4 border-b border-white/5"><div className="skeleton h-8 w-40
rounded-lg" /></div>
      <div className="p-4 space-y-3">{[1,2,3].map(i => <SkeletonCard key={i} />)}
</div>
    </div>
  );

  if (!room) return null;

  const addedCount = items.filter(i => i.status === 'added').length;
  const pendingCount = items.filter(i => i.status === 'pending').length;
  const oosCount = items.filter(i => i.status === 'oos').length;
  const isOpen = room.status === 'open';
  const isCaptain = user?.id === room.captainUid;

  const platformEmoji: Record<string, string> = {
    Blinkit: '🟡', Zepto: '🟣', 'Swiggy Instamart': '🟠',
  };

  return (
    <div className="min-h-screen flex flex-col">
      {/* Nav */}
      <nav className="flex items-center justify-between px-4 sm:px-6 py-4 border-b
border-white/5 sticky top-0 bg-gray-950/80 backdrop-blur-xl z-10">
        <div className="flex items-center gap-3">
          <Link to="/" className="text-white/40 hover:text-white transition-colors
p-1 rounded-lg hover:bg-white/10">
            <ArrowLeft className="w-4 h-4" />
          </Link>
          <div>
            <div className="flex items-center gap-2">
              <span className="text-white font-bold">{room.building}</span>
              <StatusBadge status={room.status} />
            </div>
            <p className="text-white/40 text-xs">{platformEmoji[room.platform]}
{room.platform} · Captain: {room.captainName}</p>
          </div>
        </div>
        <div className="flex items-center gap-2">
          {isOpen && (
            <CountdownTimer expiryTime={room.expiryTime} />
          )}
        </div>
      </nav>

      <div className="flex-1 max-w-4xl mx-auto w-full px-4 sm:px-6 py-6 space-y-5">

        {/* Room code share card */}
        <div className="card p-4 flex flex-col sm:flex-row items-start sm:items-
center gap-4">
```

```
          <div className="flex-1">
            <p className="text-white/40 text-xs font-medium mb-1">ROOM CODE — Share
with your group</p>
            <div className="flex items-center gap-3">
              <span className="text-4xl font-black text-white tracking-widest font-
mono">{room.roomCode}</span>
              <button id="copy-code-btn" onClick={copyCode} className="p-2 rounded-
lg hover:bg-white/10 text-white/40 hover:text-white transition-all">
                {copied ? <Check className="w-4 h-4 text-emerald-400" /> : <Copy
className="w-4 h-4" />}
              </button>
              <button id="share-btn" onClick={shareRoom} className="p-2 rounded-lg
hover:bg-white/10 text-white/40 hover:text-white transition-all">
                <Share2 className="w-4 h-4" />
              </button>
            </div>
          </div>
          <div className="flex gap-2">
            <div className="text-center px-3 py-1.5 rounded-lg bg-white/5 border
border-white/8">
              <div className="text-lg font-black text-white">{room.memberCount}
</div>
              <div className="text-white/40 text-[10px]">Members</div>
            </div>
            <div className="text-center px-3 py-1.5 rounded-lg bg-white/5 border
border-white/8">
              <div className="text-lg font-black text-white">₹{room.totalCartValue}
</div>
              <div className="text-white/40 text-[10px]">Cart</div>
            </div>
          </div>
        </div>

        {/* Progress bar */}
        <ProgressBar value={room.totalCartValue} threshold={room.threshold} />

        {/* Captain can also add their own items */}
        {isOpen && (
          <button
            id="cap-add-item-btn"
            onClick={() => setShowForm(true)}
            className="btn-primary w-full flex items-center justify-center gap-2 py-
3.5"
          >
            <Plus className="w-4 h-4" /> Add My Own Items
          </button>
        )}

        {/* Status controls */}
        {isCaptain && (
          <div className="card p-4 space-y-3">
            <p className="text-white/40 text-xs font-semibold uppercase tracking-
```

```
wider">Captain Controls</p>
            <div className="grid grid-cols-2 sm:grid-cols-4 gap-2">
              {room.status === 'open' && (
                <button id="lock-room-btn" onClick={() => updateStatus('locked')}
                  className="flex items-center justify-center gap-1.5 py-2.5 px-3
rounded-xl bg-amber-500/20 border border-amber-500/30 text-amber-400 text-sm font-
semibold hover:bg-amber-500/30 transition-all">
                  <Lock className="w-3.5 h-3.5" /> Lock Room
                </button>
              )}
              {room.status === 'locked' && (
                <button id="ordering-btn" onClick={() => updateStatus('ordering')}
                  className="flex items-center justify-center gap-1.5 py-2.5 px-3
rounded-xl bg-indigo-500/20 border border-indigo-500/30 text-indigo-400 text-sm
font-semibold hover:bg-indigo-500/30 transition-all">
                  <ShoppingBag className="w-3.5 h-3.5" /> Mark Ordered
                </button>
              )}
              {room.status === 'ordering' && (
                <button id="delivered-btn" onClick={() => updateStatus('delivered')}
                  className="flex items-center justify-center gap-1.5 py-2.5 px-3
rounded-xl bg-emerald-500/20 border border-emerald-500/30 text-emerald-400 text-sm
font-semibold hover:bg-emerald-500/30 transition-all">
                  <CheckCircle className="w-3.5 h-3.5" /> Delivered
                </button>
              )}
              {room.status === 'locked' && (
                <Link id="splits-btn" to={`/splits/${roomCode}`}
                  className="flex items-center justify-center gap-1.5 py-2.5 px-3
rounded-xl bg-purple-500/20 border border-purple-500/30 text-purple-400 text-sm
font-semibold hover:bg-purple-500/30 transition-all">
                  <Users className="w-3.5 h-3.5" /> View Splits
                </Link>
              )}
              {(room.status === 'open' || room.status === 'locked') && (
                <button
                  id="dissolve-room-btn"
                  onClick={() => {
                    if (confirm('Dissolve this room? This cannot be undone.'))
updateStatus('dissolved');
                  }}
                  className="flex items-center justify-center gap-1.5 py-2.5 px-3
rounded-xl bg-red-500/10 border border-red-500/20 text-red-400/70 text-sm font-
semibold hover:bg-red-500/20 hover:border-red-500/40 hover:text-red-400 transition-
all"
                >
                  <Trash2 className="w-3.5 h-3.5" /> Dissolve
                </button>
              )}
            </div>
          </div>
        )}
```

```
        {/* Item stats */}
        <div className="grid grid-cols-3 gap-3">
          {[
            { label: 'Pending', count: pendingCount, color: 'text-yellow-400' },
            { label: 'Added',   count: addedCount,   color: 'text-emerald-400' },
            { label: 'OOS',     count: oosCount,     color: 'text-red-400' },
          ].map(s => (
            <div key={s.label} className="card p-3 text-center">
              <div className={`text-2xl font-black ${s.color}`}>{s.count}</div>
              <div className="text-white/40 text-xs">{s.label}</div>
            </div>
          ))}
        </div>

        {/* Item feed */}
        <div className="space-y-3">
          <h2 className="text-white font-bold flex items-center gap-2">
            <Package className="w-4 h-4 text-indigo-400" />
            Items ({items.length})
          </h2>

          {items.length === 0
            ? <EmptyState icon="📦" title="No items yet" description="Members will
submit their items once they join." />
            : items.map(item => (
                <div key={item.id} className={`card p-4 space-y-2 border transition-
all ${
                  item.status === 'added' ? 'border-emerald-500/30 bg-emerald-500/5'
:
                  item.status === 'oos'   ? 'border-red-500/30 bg-red-500/5' :
                  'border-white/8'
                }`}>
                  <div className="flex items-start justify-between gap-2">
                    <div className="flex-1 min-w-0">
                      <div className="flex items-center gap-2 flex-wrap mb-0.5">
                        <span className="text-indigo-300 text-xs font-semibold">
{item.memberName}</span>
                        {item.status === 'added'   && <span className="chip-added">✓
ADDED</span>}
                        {item.status === 'oos'     && <span className="chip-
oos">OOS</span>}
                        {item.status === 'pending' && <span className="chip-
pending">PENDING</span>}
                      </div>
                      <p className="text-white font-semibold truncate">
{item.itemName}</p>
                      <p className="text-white/50 text-sm">
                        ₹{item.price} × {item.quantity} = <span className="text-
white font-medium">₹{item.subtotal}</span>
                        <span className={`ml-2 text-xs ${item.oosPreference ===
'substitute' ? 'text-amber-400' : 'text-red-400'}`}>
```

```
                          {item.oosPreference === 'substitute' ? '↔ Sub' : '× Cancel
if OOS'}
                        </span>
                      </p>
                    </div>

                    {isCaptain && (
                      <div className="flex gap-1 flex-shrink-0">
                        <button
                          onClick={() => toggleItemStatus(item)}
                          className={`p-1.5 rounded-lg border transition-all text-xs
font-semibold ${
                            item.status === 'added'
                              ? 'bg-emerald-500/20 border-emerald-500/30 text-
emerald-400 hover:bg-red-500/20 hover:border-red-500/30 hover:text-red-400'
                              : item.status === 'oos'
                              ? 'bg-red-500/20 border-red-500/30 text-red-400
hover:bg-white/10 hover:border-white/20 hover:text-white/60'
                              : 'bg-white/5 border-white/10 text-white/40 hover:bg-
emerald-500/20 hover:border-emerald-500/30 hover:text-emerald-400'
                          }`}
                          title={item.status === 'pending' ? 'Mark as Added' :
item.status === 'added' ? 'Mark as OOS' : 'Reset to Pending'}
                        >
                          {item.status === 'added' ? <Check className="w-4 h-4" /> :
item.status === 'oos' ? <X className="w-4 h-4" /> : <Check className="w-4 h-4" />}
                        </button>
                      </div>
                    )}
                  </div>

                  <a
                    href={item.itemLink}
                    target="_blank"
                    rel="noopener noreferrer"
                    className="flex items-center gap-1.5 text-indigo-400 hover:text-
indigo-300 text-xs group transition-colors"
                  >
                    <ExternalLink className="w-3 h-3 flex-shrink-0" />
                    <span className="truncate group-hover:underline">{item.itemLink}
</span>
                  </a>
                </div>
              ))
          }
        </div>

        {/* Delivered state */}
        {room.status === 'delivered' && (
          <div className="card p-6 text-center space-y-3 border-emerald-500/20 bg-
emerald-500/5">
            <div className="text-4xl">🎉</div>
```

```
            <h3 className="text-white font-bold text-lg">Order Delivered!</h3>
            <p className="text-white/50 text-sm">
              Hand off the items and collect payments from members.
            </p>
            <Link to={`/splits/${roomCode}`} className="btn-primary inline-flex
items-center gap-2">
              <Users className="w-4 h-4" /> View Payment Splits
            </Link>
          </div>
        )}
      </div>

      {showForm && (
        <AddItemForm
          onClose={() => setShowForm(false)}
          onSubmit={handleAddItem}
          allItems={items}
        />
      )}
    </div>
  );
};
\n```\n\n### src/pages/CreateRoomPage.tsx

```\nimport React, { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import {
  ArrowLeft, Building2, Zap, Clock, CreditCard, ChevronRight, Loader2,
  MapPin, Navigation, CheckCircle2,
} from 'lucide-react';
import { supabase } from '../supabase';
import { useAuth } from '../AuthContext';
import type { Platform } from '../types';
import { generateRoomCode } from '../utils';
import toast from 'react-hot-toast';

const PLATFORMS: Platform[] = ['Blinkit', 'Zepto', 'Swiggy Instamart'];
const PLATFORM_EMOJI: Record<Platform, string> = {
  Blinkit: '🟡',
  Zepto: '🟣',
  'Swiggy Instamart': '🟠',
};
const TIMERS = [10, 15, 20];

// ── Nominatim reverse-geocode ──────────────────────────────

interface NominatimResult {
  address: {
    building?:   string;
    amenity?:   string;
    leisure?:   string;
    tourism?:   string;
```
```

```
    residential?:  string;
    neighbourhood?: string;
    suburb?:       string;
    road?:         string;
    house_number?: string;
    postcode?:     string;
    city?:         string;
    town?:         string;
    village?:      string;
  };
  display_name: string;
}

async function reverseGeocode(lat: number, lon: number): Promise<string[]> {
  const url = `https://nominatim.openstreetmap.org/reverse?
lat=${lat}&lon=${lon}&format=json&zoom=18&addressdetails=1`;
  const res  = await fetch(url, { headers: { 'Accept-Language': 'en' } });
  if (!res.ok) throw new Error('Geocoding request failed');
  const data: NominatimResult = await res.json();
  const a = data.address;

  const suggestions = new Set<string>();

  // Priority order: most specific → most general
  if (a.building)      suggestions.add(a.building);
  if (a.amenity)       suggestions.add(a.amenity);
  if (a.leisure)       suggestions.add(a.leisure);
  if (a.tourism)       suggestions.add(a.tourism);
  if (a.residential)   suggestions.add(a.residential);
  if (a.neighbourhood) suggestions.add(a.neighbourhood);
  if (a.suburb)        suggestions.add(a.suburb);

  // Road + house number combo
  if (a.road && a.house_number) suggestions.add(`${a.house_number} ${a.road}`);
  else if (a.road)              suggestions.add(a.road);

  // City/ town fallback
  const city = a.city || a.town || a.village;
  if (city) suggestions.add(city);

  return [...suggestions].slice(0, 5);
}

// ── Component ──────────────────────────────────────────────

export const CreateRoomPage: React.FC = () => {
  const navigate = useNavigate();
  const { user, displayName } = useAuth();

  const [building, setBuilding]         = useState('');
  const [platform, setPlatform]         = useState<Platform>('Blinkit');
  const [timerMinutes, setTimerMinutes] = useState(15);
```

```
  const [threshold, setThreshold]       = useState(200);
  const [upiId, setUpiId]               = useState('');
  const [loading, setLoading]           = useState(false);

  // Location detection state
  const [detecting, setDetecting]         = useState(false);
  const [suggestions, setSuggestions]     = useState<string[]>([]);
  const [locationUsed, setLocationUsed]   = useState(false);

  // ── Detect Location ──────────────────────────────────────

  const handleDetectLocation = () => {
    if (!('geolocation' in navigator)) {
      toast.error('Location not supported by your browser');
      return;
    }

    setDetecting(true);
    setSuggestions([]);
    toast('📍 Detecting your location…', { duration: 2000 });

    navigator.geolocation.getCurrentPosition(
      async ({ coords }) => {
        try {
          const results = await reverseGeocode(coords.latitude, coords.longitude);
          if (results.length === 0) {
            toast.error('No nearby buildings found. Please type manually.');
          } else {
            setSuggestions(results);
            // Auto-pick the first suggestion
            setBuilding(results[0]);
            setLocationUsed(true);
            toast.success('Location detected!');
          }
        } catch {
          toast.error('Could not fetch address. Please type manually.');
        } finally {
          setDetecting(false);
        }
      },
      (err) => {
        setDetecting(false);
        if (err.code === err.PERMISSION_DENIED) {
          toast.error('Location permission denied. Please type your building
name.');
        } else {
          toast.error('Could not get location. Please type manually.');
        }
      },
      { timeout: 10000, enableHighAccuracy: true }
    );
  };
```

```
  // ── Create Room ──────────────────────────────────────────

  const handleCreate = async () => {
    if (!user) { toast.error('Not authenticated yet, please wait'); return; }
    if (!building.trim()) { toast.error('Enter your building / hostel name');
return; }
    if (!upiId.trim())    { toast.error('Enter your UPI ID to receive payments');
return; }

    setLoading(true);
    try {
      const roomCode   = generateRoomCode();
      const now        = Date.now();
      const expiryTime = now + timerMinutes * 60 * 1000;

      const { error } = await supabase.from('rooms').insert({
        room_code:        roomCode,
        captain_uid:      user.id,
        captain_name:     displayName || 'Captain',
        captain_upi:      upiId.trim(),
        building:         building.trim(),
        platform,
        status:           'open',
        expiry_time:      expiryTime,
        threshold,
        delivery_fee:     30,
        total_cart_value: 0,
        member_count:     1,
        created_at:       now,
      });

      if (error) throw error;
      toast.success(`Room ${roomCode} created!`);
      navigate(`/captain/${roomCode}`);
    } catch (err: unknown) {
      const msg = (err as { message?: string })?.message ?? String(err);
      console.error('Create room error:', err);
      toast.error(`Error: ${msg}`, { duration: 8000 });
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen flex flex-col">
      {/* Nav */}
      <nav className="flex items-center gap-3 px-4 sm:px-6 py-4 border-b border-
white/5 sticky top-0 bg-gray-950/80 backdrop-blur-xl z-10">
        <Link to="/" className="text-white/40 hover:text-white transition-colors p-1
rounded-lg hover:bg-white/10">
          <ArrowLeft className="w-4 h-4" />
```

```
        </Link>
        <div className="flex items-center gap-2">
          <div className="w-7 h-7 rounded-lg bg-indigo-600 flex items-center
justify-center">
            <Zap className="w-3.5 h-3.5 text-white" />
          </div>
          <span className="font-bold text-white text-sm sm:text-base">Create a
Room</span>
        </div>
      </nav>

      <div className="flex-1 flex items-start justify-center px-4 py-8">
        <div className="w-full max-w-md space-y-6">

          <div className="text-center space-y-1">
            <h1 className="text-2xl font-black text-white">Start a Group Run</h1>
            <p className="text-white/40 text-sm">Share the room code after
creating</p>
          </div>

          <div className="card p-6 space-y-5">

            {/* Building — with location detect */}
            <div className="space-y-1.5">
              <label className="flex items-center gap-2 text-white/60 text-sm font-
medium">
                <Building2 className="w-3.5 h-3.5" /> Building / Hostel Name
              </label>

              <div className="flex gap-2">
                <div className="relative flex-1">
                  {locationUsed && (
                    <CheckCircle2 className="absolute left-3 top-1/2 -translate-y-
1/2 w-4 h-4 text-emerald-400" />
                  )}
                  <input
                    id="building-input"
                    className={`input-field w-full transition-all ${locationUsed ?
'pl-9' : ''}`}
                    placeholder="e.g. Hostel Block C, Tower 4"
                    value={building}
                    onChange={e => { setBuilding(e.target.value);
setLocationUsed(false); }}
                  />
                </div>

                {/* Detect Location button */}
                <button
                  id="detect-location-btn"
                  onClick={handleDetectLocation}
                  disabled={detecting}
                  title="Auto-detect from GPS"
```

```
                  className="flex-shrink-0 flex items-center justify-center gap-1.5
px-3 py-2.5 rounded-xl border border-indigo-500/40 bg-indigo-600/15 text-indigo-300
hover:bg-indigo-600/30 hover:border-indigo-500/70 transition-all disabled:opacity-50
disabled:cursor-not-allowed"
                >
                  {detecting
                    ? <Loader2 className="w-4 h-4 animate-spin" />
                    : <Navigation className="w-4 h-4" />
                  }
                  <span className="text-xs font-semibold hidden sm:block">
                    {detecting ? 'Detecting…' : 'Detect'}
                  </span>
                </button>
              </div>

              {/* Location suggestions dropdown */}
              {suggestions.length > 0 && (
                <div className="mt-1.5 rounded-xl border border-white/10 bg-gray-
900/90 backdrop-blur divide-y divide-white/5 overflow-hidden shadow-xl">
                  <p className="px-3 py-2 text-[10px] text-white/30 font-semibold
uppercase tracking-wider flex items-center gap-1.5">
                    <MapPin className="w-3 h-3" /> Nearby locations — tap to select
                  </p>
                  {suggestions.map((s, i) => (
                    <button
                      key={i}
                      onClick={() => {
                        setBuilding(s);
                        setLocationUsed(true);
                        setSuggestions([]);
                      }}
                      className={`w-full text-left px-4 py-2.5 text-sm flex items-
center gap-2 transition-colors hover:bg-white/5 ${
                        building === s ? 'text-indigo-300 font-medium' : 'text-
white/70'
                      }`}
                    >
                      {building === s
                        ? <CheckCircle2 className="w-3.5 h-3.5 text-indigo-400 flex-
shrink-0" />
                        : <MapPin className="w-3.5 h-3.5 text-white/20 flex-shrink-
0" />
                      }
                      {s}
                    </button>
                  ))}
                  <button
                    onClick={() => setSuggestions([])}
                    className="w-full text-center px-3 py-2 text-xs text-white/30
hover:text-white/50 transition-colors"
                  >
                    Type manually instead
```

```
                </button>
              </div>
            )}

            <p className="text-white/25 text-xs flex items-center gap-1">
              <Navigation className="w-3 h-3" />
              Tap "Detect" to auto-fill from your GPS location
            </p>
          </div>

          {/* Platform */}
          <div className="space-y-1.5">
            <label className="flex items-center gap-2 text-white/60 text-sm font-
medium">
              <Zap className="w-3.5 h-3.5" /> Platform
            </label>
            <div className="grid grid-cols-3 gap-2">
              {PLATFORMS.map(p => (
                <button
                  key={p}
                  id={`platform-${p.toLowerCase().replace(/\s+/g, '-')}`}
                  onClick={() => setPlatform(p)}
                  className={`p-3 rounded-xl border text-center transition-all
duration-200 ${
                    platform === p
                      ? 'bg-indigo-600/20 border-indigo-500 text-white shadow-lg
shadow-indigo-500/10'
                      : 'border-white/10 text-white/50 hover:border-white/20
hover:text-white/70'
                  }`}
                >
                  <div className="text-2xl mb-1">{PLATFORM_EMOJI[p]}</div>
                  <div className="text-xs font-semibold">{p}</div>
                </button>
              ))}
            </div>
          </div>

          {/* Timer */}
          <div className="space-y-1.5">
            <label className="flex items-center gap-2 text-white/60 text-sm font-
medium">
              <Clock className="w-3.5 h-3.5" /> Collection Window
            </label>
            <div className="grid grid-cols-3 gap-2">
              {TIMERS.map(t => (
                <button
                  key={t}
                  id={`timer-${t}`}
                  onClick={() => setTimerMinutes(t)}
                  className={`py-2.5 rounded-xl border text-sm font-semibold
transition-all duration-200 ${
```

```
                      timerMinutes === t
                        ? 'bg-indigo-600 border-indigo-500 text-white'
                        : 'border-white/10 text-white/50 hover:border-white/20
hover:text-white/70'
                    }`}
                  >
                    {t} min
                  </button>
                ))}
              </div>
            </div>

            {/* Threshold */}
            <div className="space-y-1.5">
              <label className="flex items-center gap-2 text-white/60 text-sm font-
medium">
                <CreditCard className="w-3.5 h-3.5" /> Free Delivery Threshold
              </label>
              <div className="relative">
                <span className="absolute left-3.5 top-1/2 -translate-y-1/2 text-
white/30 font-medium">₹</span>
                <input
                  id="threshold-input"
                  type="number"
                  className="input-field pl-7"
                  value={threshold}
                  onChange={e => setThreshold(Number(e.target.value))}
                  min={0}
                />
              </div>
              <p className="text-white/30 text-xs">
                Orders above this amount get free delivery (otherwise ₹30 split
among members)
              </p>
            </div>

            {/* UPI ID */}
            <div className="space-y-1.5">
              <label className="flex items-center gap-2 text-white/60 text-sm font-
medium">
                <CreditCard className="w-3.5 h-3.5" /> Your UPI ID{' '}
                <span className="text-white/30 text-xs font-normal">(for receiving
payments)</span>
              </label>
              <input
                id="upi-input"
                className="input-field"
                placeholder="yourname@upi"
                value={upiId}
                onChange={e => setUpiId(e.target.value)}
              />
            </div>
```

```
          </div>

          <button
            id="create-room-btn"
            onClick={handleCreate}
            disabled={loading}
            className="btn-primary w-full flex items-center justify-center gap-2 py-
4 text-base"
          >
            {loading ? (
              <Loader2 className="w-5 h-5 animate-spin" />
            ) : (
              <>
                <Zap className="w-4 h-4" />
                Create Room
                <ChevronRight className="w-4 h-4" />
              </>
            )}
          </button>
          <p className="text-center text-white/30 text-xs">Your group will use the
room code to join</p>
        </div>
      </div>
    </div>
  );
};
\n```\n\n### src/pages/DemoPage.tsx

```\nimport React from 'react';
import { Link } from 'react-router-dom';
import {
  ArrowLeft, Zap, CheckCircle, ExternalLink,
  Users, Package, TrendingUp, Star
} from 'lucide-react';
import { DEMO_ROOM, DEMO_ITEMS, DEMO_PAYMENTS } from '../utils';
import { StatusBadge } from '../components/StatusBadge';

export const DemoPage: React.FC = () => {
  const totalItems = DEMO_ITEMS.length;
  const allPaid = DEMO_PAYMENTS.every(p => p.paymentStatus === 'paid');

  return (
    <div className="min-h-screen flex flex-col">
      {/* Nav */}
      <nav className="flex items-center justify-between px-4 sm:px-6 py-4 border-b
border-white/5 sticky top-0 bg-gray-950/80 backdrop-blur-xl z-10">
        <div className="flex items-center gap-3">
          <Link to="/" className="text-white/40 hover:text-white transition-colors
p-1 rounded-lg hover:bg-white/10">
            <ArrowLeft className="w-4 h-4" />
          </Link>
          <div className="flex items-center gap-2">
```
```

```
            <div className="w-7 h-7 rounded-lg bg-indigo-600 flex items-center
justify-center">
              <Zap className="w-3.5 h-3.5 text-white" />
            </div>
            <span className="font-bold text-white text-sm sm:text-base">Demo
Session</span>
          </div>
        </div>
        <span className="text-yellow-400 text-xs font-bold bg-yellow-400/10 border
border-yellow-400/20 px-2 py-1 rounded-full">
          ✨ DEMO
        </span>
      </nav>

      {/* Hero banner */}
      <div className="bg-gradient-to-r from-indigo-900/40 via-purple-900/20 to-
indigo-900/40 border-b border-indigo-500/20 px-6 py-8 text-center">
        <div className="max-w-xl mx-auto space-y-2">
          <p className="text-indigo-300 text-sm font-medium">Completed •
{DEMO_ROOM.building}</p>
          <h1 className="text-3xl font-black text-white">{DEMO_ROOM.platform} Group
Run</h1>
          <p className="text-white/50 text-sm">
            Captain <span className="text-indigo-300">{DEMO_ROOM.captainName}</span>
led{' '}
            {DEMO_ROOM.memberCount} members to pool ₹{DEMO_ROOM.totalCartValue}
worth of groceries —
            hitting the free delivery threshold!
          </p>
          <div className="pt-2 flex flex-wrap justify-center gap-3">
            <StatusBadge status={DEMO_ROOM.status} />
            <span className="badge bg-emerald-500/20 text-emerald-400 border border-
emerald-500/30">
              <span className="w-1.5 h-1.5 rounded-full bg-emerald-400 mr-1.5" />
              Free Delivery Unlocked
            </span>
          </div>
        </div>
      </div>

      <div className="flex-1 max-w-5xl mx-auto w-full px-4 sm:px-6 py-8 space-y-8">

        {/* Stats */}
        <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
          {[
            { label: 'Members', value: DEMO_ROOM.memberCount, icon: <Users
className="w-4 h-4" />, color: 'text-indigo-400', sub: 'joined the run' },
            { label: 'Items', value: totalItems, icon: <Package className="w-4 h-4"
/>, color: 'text-purple-400', sub: `${DEMO_ITEMS.filter(i => i.status ===
'oos').length} OOS` },
            { label: 'Cart Value', value: `₹${DEMO_ROOM.totalCartValue}`, icon:
<TrendingUp className="w-4 h-4" />, color: 'text-emerald-400', sub: 'above
```

```
threshold' },
            { label: 'All Settled', value: allPaid ? '✓' : '…', icon: <Star
className="w-4 h-4" />, color: 'text-yellow-400', sub: 'in < 2 min' },
          ].map(stat => (
            <div key={stat.label} className="card p-4 space-y-1">
              <div className={`flex items-center gap-1.5 ${stat.color}`}>
                {stat.icon}
                <span className="text-white/40 text-xs font-medium">{stat.label}
</span>
              </div>
              <div className={`text-2xl font-black ${stat.color}`}>{stat.value}
</div>
              <div className="text-white/30 text-[11px]">{stat.sub}</div>
            </div>
          ))}
        </div>

        <div className="grid lg:grid-cols-2 gap-6">

          {/* Item Feed */}
          <div className="space-y-4">
            <h2 className="text-white font-bold flex items-center gap-2">
              <Package className="w-4 h-4 text-indigo-400" />
              Item Submissions
            </h2>
            <div className="space-y-3">
              {DEMO_ITEMS.map(item => (
                <div key={item.id} className="card p-4 space-y-2.5">
                  <div className="flex items-start justify-between gap-2">
                    <div className="flex-1 min-w-0">
                      <div className="flex items-center gap-2 flex-wrap mb-0.5">
                        <span className="text-indigo-300 text-xs font-semibold">
{item.memberName}</span>
                        {item.status === 'added' && <span className="chip-added">✓
ADDED</span>}
                        {item.status === 'oos' && <span className="chip-
oos">OOS</span>}
                        {item.status === 'pending' && <span className="chip-
pending">PENDING</span>}
                      </div>
                      <p className="text-white font-semibold">{item.itemName}</p>
                      <p className="text-white/50 text-sm">
                        ₹{item.price} × {item.quantity} ={' '}
                        <span className="text-white font-medium">₹{item.subtotal}
</span>
                      </p>
                    </div>
                    <span className={`flex-shrink-0 badge text-xs
${item.oosPreference === 'substitute' ? 'bg-amber-500/15 text-amber-400 border
border-amber-500/25' : 'bg-red-500/15 text-red-400 border border-red-500/25'}`}>
                      {item.oosPreference === 'substitute' ? '↔ Sub' : '× Cancel'}
                    </span>
```

```
                  </div>
                  <a
                    href={item.itemLink}
                    target="_blank"
                    rel="noopener noreferrer"
                    className="flex items-center gap-1.5 text-indigo-400 hover:text-
indigo-300 text-xs group transition-colors"
                  >
                    <ExternalLink className="w-3 h-3 flex-shrink-0" />
                    <span className="truncate group-hover:underline">{item.itemLink}
</span>
                  </a>
                </div>
              ))}
            </div>
          </div>

          {/* Payment splits */}
          <div className="space-y-4">
            <h2 className="text-white font-bold flex items-center gap-2">
              <CheckCircle className="w-4 h-4 text-emerald-400" />
              Payment Splits
            </h2>
            <div className="card overflow-hidden">
              <table className="w-full text-sm">
                <thead>
                  <tr className="border-b border-white/8">
                    <th className="text-left px-4 py-3 text-white/40 text-xs font-
semibold uppercase tracking-wider">Member</th>
                    <th className="text-right px-3 py-3 text-white/40 text-xs font-
semibold uppercase tracking-wider">Items</th>
                    <th className="text-right px-3 py-3 text-white/40 text-xs font-
semibold uppercase tracking-wider">Total</th>
                    <th className="text-center px-4 py-3 text-white/40 text-xs font-
semibold uppercase tracking-wider">Status</th>
                  </tr>
                </thead>
                <tbody className="divide-y divide-white/5">
                  {DEMO_PAYMENTS.map(p => (
                    <tr key={p.memberUid} className="bg-emerald-500/3">
                      <td className="px-4 py-3.5">
                        <div className="flex items-center gap-2">
                          <div className="w-6 h-6 rounded-full bg-indigo-600/30
border border-indigo-500/30 flex items-center justify-center text-[10px] font-bold
text-indigo-300">
                            {p.memberName[0]}
                          </div>
                          <span className="text-white font-medium">{p.memberName}
</span>
                        </div>
                      </td>
                      <td className="px-3 py-3.5 text-right text-white/50">
```

```
{p.itemCount}</td>
                      <td className="px-3 py-3.5 text-right text-white font-bold">₹
{p.total}</td>
                      <td className="px-4 py-3.5 text-center">
                        <span className="chip-paid">✓ PAID</span>
                      </td>
                    </tr>
                  ))}
                </tbody>
                <tfoot>
                  <tr className="border-t-2 border-white/10 bg-white/3">
                    <td className="px-4 py-3.5 text-white/50 font-semibold text-xs
uppercase tracking-wider">TOTAL</td>
                    <td className="px-3 py-3.5 text-right text-white/50">
{DEMO_ITEMS.filter(i => i.status === 'added').length}</td>
                    <td className="px-3 py-3.5 text-right text-white font-black
text-base">
                      ₹{DEMO_PAYMENTS.reduce((s, p) => s + p.total, 0)}
                    </td>
                    <td className="px-4 py-3.5 text-center text-emerald-400 text-sm
font-bold">All ✓</td>
                  </tr>
                </tfoot>
              </table>
            </div>

            {/* Timeline */}
            <h2 className="text-white font-bold flex items-center gap-2 pt-2">
              <span className="text-indigo-400">⚡</span> Session Timeline
            </h2>
            <div className="card p-5 space-y-4">
              {[
                { time: '9:00 PM', event: `${DEMO_ROOM.captainName} created Room
DEMO01`, color: 'bg-indigo-500' },
                { time: '9:02 PM', event: '4 members joined via the 6-digit code',
color: 'bg-purple-500' },
                { time: '9:08 PM', event: `${DEMO_ITEMS.length} items submitted — ₹
${DEMO_ROOM.totalCartValue} cart value`, color: 'bg-emerald-500' },
                { time: '9:10 PM', event: 'Free delivery threshold crossed 🎉 Auto-
extended', color: 'bg-yellow-500' },
                { time: '9:15 PM', event: 'Captain locked room & viewed splits',
color: 'bg-amber-500' },
                { time: '9:17 PM', event: 'Order placed on Blinkit', color: 'bg-
indigo-500' },
                { time: '9:42 PM', event: 'Delivery arrived — all items handed off',
color: 'bg-emerald-500' },
                { time: '9:44 PM', event: 'All members paid via UPI ✓', color: 'bg-
emerald-500' },
              ].map((evt, i) => (
                <div key={i} className="flex items-start gap-3">
                  <div className="flex-shrink-0 flex flex-col items-center">
                    <div className={`w-2 h-2 rounded-full mt-1 ${evt.color}`} />
```

```
                    {i < 7 && <div className="w-px flex-1 min-h-[20px] bg-white/10
mt-1" />}
                  </div>
                  <div className="flex-1 pb-2">
                    <span className="text-white/30 text-xs font-mono">{evt.time}
</span>
                    <p className="text-white/70 text-sm mt-0.5">{evt.event}</p>
                  </div>
                </div>
              ))}
            </div>
          </div>
        </div>

        {/* CTA */}
        <div className="card p-6 text-center bg-gradient-to-r from-indigo-900/30 to-
purple-900/20 border-indigo-500/20 space-y-4">
          <h3 className="text-white font-bold text-xl">Ready to start your own run?
</h3>
          <p className="text-white/50 text-sm">Create a room, share the code, pool
orders — done in under 2 minutes.</p>
          <div className="flex flex-col sm:flex-row gap-3 justify-center">
            <Link to="/create" className="btn-primary inline-flex items-center
justify-center gap-2">
              <Zap className="w-4 h-4" />
              Create a Room
            </Link>
            <Link to="/join" className="btn-secondary inline-flex items-center
justify-center gap-2">
              Join a Room
            </Link>
          </div>
        </div>

      </div>
    </div>
  );
};
\n```\n\n### src/pages/HomePage.tsx

```\nimport React, { useState, useEffect } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { Zap, ChevronRight, Users, Package, CheckCircle, Star, Clock, Pencil, X }
from 'lucide-react';
import { useAuth } from '../AuthContext';
import { DEMO_ROOM, DEMO_ITEMS, DEMO_PAYMENTS } from '../utils';

const NameModal: React.FC<{ current: string; onSubmit: (name: string) => void;
onClose: () => void }> = ({ current, onSubmit, onClose }) => {
  const [name, setName] = useState(current);
  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-
```
```

```
black/70 backdrop-blur-sm">
      <div className="card modal-enter w-full max-w-sm p-8 space-y-6 text-center
relative">
        <button onClick={onClose} className="absolute top-4 right-4 text-white/30
hover:text-white transition-colors"><X className="w-5 h-5" /></button>
        <div className="w-16 h-16 rounded-2xl bg-indigo-600/20 border border-indigo-
500/30 flex items-center justify-center mx-auto">
          <Zap className="w-8 h-8 text-indigo-400" />
        </div>
        <div>
          <h2 className="text-xl font-bold text-white">{current ? 'Change name' :
'Hey there! 👋'}</h2>
          <p className="text-white/50 text-sm mt-1">What should your group call you?
</p>
        </div>
        <div>
          <input
            className="input-field text-center text-lg font-semibold"
            placeholder="Your name..."
            value={name}
            onChange={e => setName(e.target.value)}
            onKeyDown={e => e.key === 'Enter' && name.trim() &&
onSubmit(name.trim())}
            autoFocus
          />
        </div>
        <button
          className="btn-primary w-full"
          disabled={!name.trim()}
          onClick={() => onSubmit(name.trim())}
        >
          {current ? 'Save →' : "Let's Go →"}
        </button>
      </div>
    </div>
  );
};

const DemoCard: React.FC = () => {
  const navigate = useNavigate();
  const totalValue = DEMO_PAYMENTS.reduce((s, p) => s + p.total, 0);
  return (
    <div
      className="card-hover p-5 cursor-pointer border-indigo-500/20 hover:border-
indigo-400/40 transition-all"
      onClick={() => navigate('/demo')}
    >
      <div className="flex items-start justify-between mb-4">
        <div>
          <div className="flex items-center gap-2 mb-1">
            <span className="text-yellow-400 text-xs font-bold bg-yellow-400/10
border border-yellow-400/20 px-2 py-0.5 rounded-full">
```

```
              ✨ DEMO SESSION
            </span>
          </div>
          <h3 className="font-bold text-white">{DEMO_ROOM.building}</h3>
          <p className="text-white/50 text-sm">Captain: {DEMO_ROOM.captainName} •
{DEMO_ROOM.platform}</p>
        </div>
        <span className="badge-delivered">DELIVERED</span>
      </div>

      <div className="grid grid-cols-4 gap-2 mb-4">
        {[
          { label: 'Members', value: DEMO_ROOM.memberCount, icon: <Users
className="w-3 h-3" /> },
          { label: 'Items', value: DEMO_ITEMS.filter(i => i.status !==
'oos').length, icon: <Package className="w-3 h-3" /> },
          { label: 'Cart', value: `₹${DEMO_ROOM.totalCartValue}`, icon: <Star
className="w-3 h-3" /> },
          { label: 'All Paid', value: '✓', icon: <CheckCircle className="w-3 h-3" />
},
        ].map(stat => (
          <div key={stat.label} className="bg-white/5 rounded-xl p-2.5 text-center">
            <div className="text-white/40 flex justify-center mb-0.5">{stat.icon}
</div>
            <div className="text-white font-bold text-sm">{stat.value}</div>
            <div className="text-white/40 text-[10px]">{stat.label}</div>
          </div>
        ))}
      </div>

      <div className="flex items-center justify-between text-sm">
        <span className="text-white/50">Total settled: <span className="text-
emerald-400 font-semibold">₹{totalValue}</span></span>
        <div className="flex items-center gap-1 text-indigo-400 font-medium">
          <span>View full session</span>
          <ChevronRight className="w-4 h-4" />
        </div>
      </div>
    </div>
  );
};

// ── Recent Rooms panel ──────────────────────────────────────────

const PLATFORM_EMOJI: Record<string, string> = {
  Blinkit: '🟡', Zepto: '🟣', 'Swiggy Instamart': '🟠',
};

interface HistoryEntry {
  roomCode: string;
  building: string;
  platform: string;
```

```
  role: string;
  visitedAt: number;
}

const RecentRooms: React.FC = () => {
  const [history, setHistory] = useState<HistoryEntry[]>([]);

  useEffect(() => {
    const raw = localStorage.getItem('droprun_room_history');
    if (raw) setHistory(JSON.parse(raw));
  }, []);

  if (history.length === 0) return null;

  const formatTime = (ts: number) => {
    const diff = Date.now() - ts;
    const mins = Math.floor(diff / 60000);
    if (mins < 1) return 'Just now';
    if (mins < 60) return `${mins}m ago`;
    const hrs = Math.floor(mins / 60);
    if (hrs < 24) return `${hrs}h ago`;
    return `${Math.floor(hrs / 24)}d ago`;
  };

  return (
    <div className="w-full max-w-lg mb-6">
      <p className="text-white/40 text-xs font-semibold uppercase tracking-widest
mb-3 text-center">Recent Rooms</p>
      <div className="space-y-2">
        {history.slice(0, 3).map(entry => (
          <Link
            key={entry.roomCode}
            to={entry.role === 'captain' ? `/captain/${entry.roomCode}` :
`/member/${entry.roomCode}`}
            className="flex items-center gap-3 card p-3 hover:bg-white/10
hover:border-white/20 transition-all group"
          >
            <div className="w-8 h-8 rounded-lg bg-white/5 border border-white/10
flex items-center justify-center text-base flex-shrink-0">
              {PLATFORM_EMOJI[entry.platform] || '📦'}
            </div>
            <div className="flex-1 min-w-0">
              <div className="flex items-center gap-2">
                <p className="text-white text-sm font-semibold truncate">
{entry.building}</p>
                <span className={`text-[10px] font-bold px-1.5 py-0.5 rounded-full
${
                  entry.role === 'captain' ? 'bg-indigo-500/20 text-indigo-300' :
'bg-white/10 text-white/50'
                }`}>{entry.role === 'captain' ? 'Captain' : 'Member'}</span>
              </div>
              <div className="flex items-center gap-2 text-xs text-white/30">
```

```
                <Clock className="w-3 h-3" />
                <span>{formatTime(entry.visitedAt)}</span>
                <span className="font-mono">· #{entry.roomCode}</span>
              </div>
            </div>
            <ChevronRight className="w-4 h-4 text-white/20 group-hover:text-white/50
transition-colors" />
          </Link>
        ))}
      </div>
    </div>
  );
};

export const HomePage: React.FC = () => {
  const { displayName, setDisplayName } = useAuth();
  const [showNameModal, setShowNameModal] = useState(!displayName);
  const [editingName, setEditingName] = useState(false);
  const navigate = useNavigate();

  const handleNameSubmit = (name: string) => {
    setDisplayName(name);
    setShowNameModal(false);
    setEditingName(false);
  };

  const handleCreate = () => {
    if (!displayName) { setShowNameModal(true); return; }
    navigate('/create');
  };

  const handleJoin = () => {
    if (!displayName) { setShowNameModal(true); return; }
    navigate('/join');
  };

  return (
    <>
      {(showNameModal || editingName) && (
        <NameModal
          current={editingName ? displayName : ''}
          onSubmit={handleNameSubmit}
          onClose={() => { setShowNameModal(false); setEditingName(false); }}
        />
      )}

      <div className="min-h-screen flex flex-col">
        {/* Top nav */}
        <nav className="flex items-center justify-between px-6 py-4 border-b border-
white/5">
          <div className="flex items-center gap-2">
            <div className="w-8 h-8 rounded-xl bg-indigo-600 flex items-center
```

```
justify-center">
              <Zap className="w-4 h-4 text-white" />
            </div>
            <span className="font-bold text-white text-lg tracking-
tight">DropRun</span>
          </div>
          {displayName && (
            <button
              onClick={() => setEditingName(true)}
              className="flex items-center gap-2 bg-white/5 hover:bg-white/10
rounded-full px-3 py-1.5 border border-white/10 hover:border-white/20 transition-all
group"
            >
              <div className="w-5 h-5 rounded-full bg-indigo-500 flex items-center
justify-center text-[10px] font-bold">
                {displayName[0].toUpperCase()}
              </div>
              <span className="text-white/70 text-sm">{displayName}</span>
              <Pencil className="w-3 h-3 text-white/30 group-hover:text-white/60
transition-colors" />
            </button>
          )}
        </nav>

        <main className="flex-1 flex flex-col items-center justify-center px-4 py-
12">
          {/* Hero */}
          <div className="text-center space-y-4 mb-12 max-w-lg">
            <div className="inline-flex items-center gap-2 bg-indigo-500/10 border
border-indigo-500/30 rounded-full px-4 py-1.5 text-indigo-300 text-sm font-medium
mb-2">
              <Zap className="w-3.5 h-3.5" />
              Hyper-local order pooling
            </div>
            <h1 className="text-4xl sm:text-5xl font-black text-white leading-
tight">
              Order Together,{' '}
              <span className="bg-gradient-to-r from-indigo-400 to-purple-400 bg-
clip-text text-transparent">
                Save Together
              </span>
            </h1>
            <p className="text-white/50 text-lg leading-relaxed">
              Pool your Blinkit, Zepto & Instamart orders with your hostel mates.
              One Captain, one delivery, zero individual delivery fees.
            </p>
          </div>

          {/* Action cards */}
          <div className="grid sm:grid-cols-2 gap-4 w-full max-w-lg mb-12">
            {/* Captain */}
            <button
```

```
              onClick={handleCreate}
              className="group relative overflow-hidden bg-gradient-to-br from-
indigo-600 to-indigo-800 hover:from-indigo-500 hover:to-indigo-700 rounded-2xl p-6
text-left transition-all duration-300 hover:-translate-y-1 hover:shadow-2xl
hover:shadow-indigo-900/60 border border-indigo-500/30"
            >
              <div className="absolute top-0 right-0 w-24 h-24 bg-white/5 rounded-
full -translate-y-6 translate-x-6 group-hover:scale-150 transition-transform
duration-500" />
              <div className="w-10 h-10 rounded-xl bg-white/15 flex items-center
justify-center mb-4">
                <Zap className="w-5 h-5 text-white" />
              </div>
              <h2 className="text-white font-bold text-xl mb-1">Create Room</h2>
              <p className="text-indigo-200/70 text-sm leading-relaxed">
                Be the Captain. Invite your group and manage the order.
              </p>
              <div className="flex items-center gap-1 mt-4 text-indigo-300 text-sm
font-medium group-hover:gap-2 transition-all">
                Start a run <ChevronRight className="w-4 h-4" />
              </div>
            </button>

            {/* Member */}
            <button
              onClick={handleJoin}
              className="group relative overflow-hidden bg-white/5 hover:bg-white/10
rounded-2xl p-6 text-left transition-all duration-300 hover:-translate-y-1
hover:shadow-2xl hover:shadow-white/5 border border-white/10 hover:border-white/20"
            >
              <div className="absolute top-0 right-0 w-24 h-24 bg-white/3 rounded-
full -translate-y-6 translate-x-6 group-hover:scale-150 transition-transform
duration-500" />
              <div className="w-10 h-10 rounded-xl bg-white/10 flex items-center
justify-center mb-4">
                <Users className="w-5 h-5 text-white/70" />
              </div>
              <h2 className="text-white font-bold text-xl mb-1">Join Room</h2>
              <p className="text-white/40 text-sm leading-relaxed">
                Got a Room Code? Jump in and add your items.
              </p>
              <div className="flex items-center gap-1 mt-4 text-white/50 text-sm
font-medium group-hover:text-white/70 group-hover:gap-2 transition-all">
                Enter code <ChevronRight className="w-4 h-4" />
              </div>
            </button>
          </div>

          {/* How it works */}
          <div className="w-full max-w-lg mb-8">
            <div className="grid grid-cols-3 gap-3">
              {[
```

```
                { step: '1', text: 'Captain creates a room & shares the 6-digit
code', color: 'text-indigo-400' },
                { step: '2', text: 'Members join, paste item links & quantities',
color: 'text-emerald-400' },
                { step: '3', text: 'Captain orders, splits are calculated
automatically', color: 'text-purple-400' },
              ].map(({ step, text, color }) => (
                <div key={step} className="glass-panel rounded-xl p-3 text-center">
                  <div className={`text-2xl font-black mb-1.5 ${color}`}>{step}
</div>
                  <p className="text-white/50 text-[11px] leading-snug">{text}</p>
                </div>
              ))}
            </div>
          </div>

          {/* Demo section */}
          <div className="w-full max-w-lg">
            <p className="text-white/40 text-xs font-semibold uppercase tracking-
widest mb-3 text-center">
              See how it works
            </p>
            <DemoCard />
          </div>

          {/* Recent rooms */}
          <RecentRooms />
        </main>

        <footer className="text-center py-4 text-white/20 text-xs border-t border-
white/5">
          DropRun — Zero-friction order pooling for your building
        </footer>
      </div>
    </>
  );
};
\n```\n\n### src/pages/JoinRoomPage.tsx

```\nimport React, { useState, useEffect, useCallback } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import {
  ArrowLeft, Zap, Loader2, MapPin, Navigation, Users,
  Clock, RefreshCw, ChevronRight, Search, ArrowRightLeft,
  CheckCircle2, X,
} from 'lucide-react';
import { supabase } from '../supabase';
import { useAuth } from '../AuthContext';
import type { Room } from '../types';
import type { Platform, RoomStatus } from '../types';
import { CountdownTimer } from '../components/CountdownTimer';
import toast from 'react-hot-toast';
```
```

```
// ── DB mapper ──────────────────────────────────────────────

const mapRoom = (r: Record<string, unknown>): Room => ({
  roomCode:       r.room_code as string,
  captainUid:     r.captain_uid as string,
  captainName:    r.captain_name as string,
  captainUPI:     r.captain_upi as string,
  building:       r.building as string,
  platform:       r.platform as Platform,
  status:         r.status as RoomStatus,
  expiryTime:     r.expiry_time as number,
  threshold:      r.threshold as number,
  deliveryFee:    r.delivery_fee as number,
  totalCartValue: r.total_cart_value as number,
  memberCount:    r.member_count as number,
  createdAt:      r.created_at as number,
});

// ── Nominatim reverse-geocode ──────────────────────────────

async function reverseGeocode(lat: number, lon: number): Promise<string[]> {
  const url = `https://nominatim.openstreetmap.org/reverse?
lat=${lat}&lon=${lon}&format=json&zoom=16&addressdetails=1`;
  const res  = await fetch(url, { headers: { 'Accept-Language': 'en' } });
  if (!res.ok) throw new Error('Geocoding failed');
  const data = await res.json();
  const a = data.address ?? {};

  const terms = new Set<string>();
  if (a.building)      terms.add(a.building);
  if (a.amenity)       terms.add(a.amenity);
  if (a.leisure)       terms.add(a.leisure);
  if (a.residential)   terms.add(a.residential);
  if (a.neighbourhood) terms.add(a.neighbourhood);
  if (a.suburb)        terms.add(a.suburb);
  if (a.road)          terms.add(a.road);
  const city = a.city || a.town || a.village;
  if (city)            terms.add(city);

  return [...terms].slice(0, 6);
}

// ── Same-building heuristic ────────────────────────────────
// Two rooms are "in the same building" if their building strings share
// at least 2 significant words, or are an exact (case-insensitive) match.

function isSameBuilding(a: string, b: string): boolean {
  const normalize = (s: string) =>
    s.toLowerCase().replace(/[^\w\s]/g, ' ').trim();

  if (normalize(a) === normalize(b)) return true;
```

```
  const sig = (s: string) =>
    new Set(normalize(s).split(/\s+/).filter(w => w.length > 2));

  const aWords = sig(a);
  const bWords = sig(b);
  const overlap = [...aWords].filter(w => bWords.has(w));
  return overlap.length >= Math.min(2, Math.min(aWords.size, bWords.size));
}

// ── Platform meta ──────────────────────────────────────────

const PLATFORM_EMOJI: Record<string, string> = {
  Blinkit: '🟡', Zepto: '🟣', 'Swiggy Instamart': '🟠',
};

const PLATFORM_COLOR: Record<string, string> = {
  Blinkit:           'text-yellow-400 bg-yellow-400/10 border-yellow-400/20',
  Zepto:             'text-purple-400 bg-purple-400/10 border-purple-400/20',
  'Swiggy Instamart':'text-orange-400 bg-orange-400/10 border-orange-400/20',
};

// ── Room card ──────────────────────────────────────────────

const RoomCard: React.FC<{
  room: Room;
  onJoin: (room: Room) => void;
  joining: boolean;
  isOverflowTarget?: boolean;   // badge: "🔀 Accepting overflow"
}> = ({ room, onJoin, joining, isOverflowTarget }) => {
  const thresholdHit = room.totalCartValue >= room.threshold;
  const progressPct  = Math.min(100, (room.totalCartValue / room.threshold) * 100);

  return (
    <div
      className={`card p-4 space-y-3 transition-all duration-300 hover:-translate-y-
0.5 hover:shadow-xl cursor-pointer group
        ${thresholdHit
          ? 'border-emerald-500/30 bg-emerald-500/5 hover:border-emerald-400/50
hover:shadow-emerald-900/30'
          : isOverflowTarget
            ? 'border-indigo-500/40 bg-indigo-500/5 hover:border-indigo-400/60
hover:shadow-indigo-900/30'
            : 'hover:border-indigo-500/40 hover:shadow-indigo-900/30'
        }`}
      onClick={() => !joining && onJoin(room)}
    >
      {/* Overflow / threshold badges */}
      {(thresholdHit || isOverflowTarget) && (
        <div className="flex items-center gap-2 flex-wrap -mb-1">
          {thresholdHit && (
            <span className="inline-flex items-center gap-1 text-[11px] font-bold
```

```
px-2 py-0.5 rounded-full bg-emerald-500/15 border border-emerald-500/30 text-
emerald-300">
              <CheckCircle2 className="w-3 h-3" /> Threshold reached · Free delivery
🎉
            </span>
          )}
          {isOverflowTarget && !thresholdHit && (
            <span className="inline-flex items-center gap-1 text-[11px] font-bold
px-2 py-0.5 rounded-full bg-indigo-500/15 border border-indigo-500/30 text-indigo-
300 animate-pulse">
              <ArrowRightLeft className="w-3 h-3" /> Accepting overflow members
            </span>
          )}
        </div>
      )}

      {/* Header row */}
      <div className="flex items-start justify-between gap-2">
        <div className="flex-1 min-w-0">
          <div className="flex items-center gap-2 flex-wrap mb-0.5">
            <span className={`inline-flex items-center gap-1 text-[11px] font-bold
px-2 py-0.5 rounded-full border ${PLATFORM_COLOR[room.platform] ?? 'text-white/50
bg-white/5 border-white/10'}`}>
              {PLATFORM_EMOJI[room.platform]} {room.platform}
            </span>
          </div>
          <p className="text-white font-bold truncate">{room.building}</p>
          <p className="text-white/40 text-xs">Captain: {room.captainName}</p>
        </div>

        <div className="flex flex-col items-end gap-1.5 flex-shrink-0">
          <CountdownTimer expiryTime={room.expiryTime} />
          <button
            id={`join-room-${room.roomCode}`}
            disabled={joining}
            onClick={e => { e.stopPropagation(); onJoin(room); }}
            className={`flex items-center gap-1.5 px-3 py-1.5 rounded-lg text-white
text-xs font-bold transition-all group-hover:gap-2 disabled:opacity-60
              ${thresholdHit
                ? 'bg-emerald-600 hover:bg-emerald-500'
                : isOverflowTarget
                  ? 'bg-indigo-500 hover:bg-indigo-400 ring-1 ring-indigo-400/50'
                  : 'bg-indigo-600 hover:bg-indigo-500'
              }`}
          >
            {joining
              ? <Loader2 className="w-3 h-3 animate-spin" />
              : <><span>Join</span><ChevronRight className="w-3 h-3" /></>
            }
          </button>
        </div>
      </div>
```

```
      {/* Stats row */}
      <div className="flex items-center gap-4 text-xs text-white/50">
        <span className="flex items-center gap-1">
          <Users className="w-3 h-3" />{room.memberCount} member{room.memberCount
!== 1 ? 's' : ''}
        </span>
        <span className="flex items-center gap-1">
          ₹{room.totalCartValue} / ₹{room.threshold}
        </span>
        <span className="font-mono text-white/30">#{room.roomCode}</span>
      </div>

      {/* Progress */}
      <div>
        <div className="flex justify-between text-[10px] text-white/30 mb-1">
          <span>{thresholdHit ? '✓ Free delivery secured' : 'towards free delivery'}
</span>
          <span>{Math.round(progressPct)}%</span>
        </div>
        <div className="h-1.5 bg-white/8 rounded-full overflow-hidden">
          <div
            className={`h-full rounded-full transition-all ${thresholdHit ? 'bg-
emerald-400' : isOverflowTarget ? 'bg-indigo-400' : 'bg-indigo-500'}`}
            style={{ width: `${progressPct}%` }}
          />
        </div>
      </div>
    </div>
  );
};

// ── Overflow Modal ──────────────────────────────────────────

interface OverflowData {
  original: Room;
  alternatives: Room[];
}

const OverflowModal: React.FC<{
  data: OverflowData;
  joining: boolean;
  onStay: () => void;
  onSwitch: (room: Room) => void;
  onClose: () => void;
}> = ({ data, joining, onStay, onSwitch, onClose }) => {
  const best = data.alternatives[0];
  const neededForFreeDelivery = Math.max(0, best.threshold - best.totalCartValue);
  const progressPct = Math.min(100, (best.totalCartValue / best.threshold) * 100);

  return (
    <div className="fixed inset-0 z-50 flex items-end sm:items-center justify-center
```

```
p-4 bg-black/75 backdrop-blur-sm">
      <div className="card w-full max-w-sm p-6 space-y-5 relative animate-
[slideUp_0.25s_ease]">

        <button
          onClick={onClose}
          className="absolute top-4 right-4 text-white/30 hover:text-white
transition-colors"
        >
          <X className="w-5 h-5" />
        </button>

        {/* Header */}
        <div className="text-center space-y-2">
          <div className="w-14 h-14 rounded-2xl bg-emerald-500/15 border border-
emerald-500/30 flex items-center justify-center mx-auto">
            <CheckCircle2 className="w-7 h-7 text-emerald-400" />
          </div>
          <h2 className="text-white font-bold text-lg leading-tight">
            Room <span className="font-mono">#{data.original.roomCode}</span> is
full!
          </h2>
          <p className="text-white/50 text-sm">
            This room has already hit its free delivery threshold.
            Another room in the same building needs members!
          </p>
        </div>

        {/* Alternative room preview */}
        <div className="rounded-2xl border border-indigo-500/30 bg-indigo-500/8 p-4
space-y-3">
          <div className="flex items-center gap-2">
            <span className="text-[11px] font-bold px-2 py-0.5 rounded-full bg-
indigo-500/20 border border-indigo-500/30 text-indigo-300 flex items-center gap-1">
              <ArrowRightLeft className="w-3 h-3" /> Suggested room
            </span>
            <span className="font-mono font-bold text-white text-sm">#
{best.roomCode}</span>
          </div>

          <div>
            <p className="text-white font-semibold text-sm">{best.building}</p>
            <p className="text-white/40 text-xs">
              {PLATFORM_EMOJI[best.platform]} {best.platform} · Captain:
{best.captainName}
            </p>
          </div>

          <div className="space-y-1">
            <div className="flex justify-between text-xs text-white/50">
              <span>₹{best.totalCartValue} cart · {best.memberCount} members</span>
              {neededForFreeDelivery > 0 && (
```

```
                <span className="text-amber-400 font-medium">₹
{neededForFreeDelivery} to free delivery</span>
              )}
            </div>
            <div className="h-1.5 bg-white/8 rounded-full overflow-hidden">
              <div
                className="h-full bg-indigo-400 rounded-full transition-all"
                style={{ width: `${progressPct}%` }}
              />
            </div>
          </div>

          <button
            onClick={() => onSwitch(best)}
            disabled={joining}
            className="btn-primary w-full flex items-center justify-center gap-2 py-
3 text-sm"
          >
            {joining
              ? <Loader2 className="w-4 h-4 animate-spin" />
              : <><ArrowRightLeft className="w-4 h-4" /> Switch to #{best.roomCode}
</>
            }
          </button>
        </div>

        {/* Stay option */}
        <button
          onClick={onStay}
          disabled={joining}
          className="w-full text-center text-white/40 hover:text-white/70 text-sm
py-2 transition-colors"
        >
          Stay in #{data.original.roomCode} anyway →
        </button>

        {/* Show more alternatives */}
        {data.alternatives.length > 1 && (
          <p className="text-center text-white/25 text-xs">
            + {data.alternatives.length - 1} more room{data.alternatives.length > 2
? 's' : ''} available in this building
          </p>
        )}
      </div>
    </div>
  );
};

// ── Main component ──────────────────────────────────────────────

export const JoinRoomPage: React.FC = () => {
  const navigate = useNavigate();
```

```
  const { user } = useAuth();

  // manual code entry
  const [code, setCode]       = useState('');
  const [joining, setJoining] = useState<string | null>(null);

  // overflow state
  const [overflowData, setOverflowData] = useState<OverflowData | null>(null);

  // location + rooms
  const [locationTerms, setLocationTerms] = useState<string[]>([]);
  const [nearbyRooms, setNearbyRooms]     = useState<Room[]>([]);
  const [otherRooms, setOtherRooms]       = useState<Room[]>([]);
  const [locStatus, setLocStatus]         = useState<'idle' | 'detecting' | 'done' |
'denied' | 'error'>('idle');
  const [roomsLoading, setRoomsLoading]   = useState(false);

  // all open rooms (flat, for building-sibling lookup)
  const [allOpenRooms, setAllOpenRooms]   = useState<Room[]>([]);

  // ── fetch all open rooms ──────────────────────────────

  const fetchAllRooms = useCallback(async (terms: string[]) => {
    setRoomsLoading(true);
    try {
      const { data, error } = await supabase
        .from('rooms')
        .select('*')
        .eq('status', 'open')
        .order('created_at', { ascending: false })
        .limit(200);

      if (error) throw error;

      const lowerTerms = terms.map(t => t.toLowerCase());
      const now = Date.now();

      const allOpen = (data ?? [])
        .map(mapRoom)
        .filter(r => r.expiryTime > now);

      setAllOpenRooms(allOpen);

      const isNearby = (r: Room) =>
        lowerTerms.length > 0 &&
        lowerTerms.some(
          t => r.building.toLowerCase().includes(t) ||
               t.includes(r.building.toLowerCase().split(/[,\s]+/)[0])
        );

      const nearby = allOpen
        .filter(isNearby)
```

```
        .sort((a, b) => (b.totalCartValue / b.threshold) - (a.totalCartValue /
a.threshold));

      const nearbySet = new Set(nearby.map(r => r.roomCode));
      const other = allOpen
        .filter(r => !nearbySet.has(r.roomCode))
        .sort((a, b) => (b.totalCartValue / b.threshold) - (a.totalCartValue /
a.threshold));

      setNearbyRooms(nearby);
      setOtherRooms(other);
    } catch (err) {
      console.error(err);
    } finally {
      setRoomsLoading(false);
    }
  }, []);

  // ── detect location on mount ──────────────────────────────

  const detectLocation = useCallback(() => {
    if (!('geolocation' in navigator)) {
      setLocStatus('error');
      fetchAllRooms([]);
      return;
    }

    setLocStatus('detecting');
    setNearbyRooms([]);
    setOtherRooms([]);
    fetchAllRooms([]);

    navigator.geolocation.getCurrentPosition(
      async ({ coords }) => {
        try {
          const terms = await reverseGeocode(coords.latitude, coords.longitude);
          setLocationTerms(terms);
          setLocStatus('done');
          await fetchAllRooms(terms);
        } catch {
          setLocStatus('error');
        }
      },
      (err) => {
        setLocStatus(err.code === err.PERMISSION_DENIED ? 'denied' : 'error');
      },
      { timeout: 10000, enableHighAccuracy: true }
    );
  }, [fetchAllRooms]);

  useEffect(() => { detectLocation(); }, [detectLocation]);
```

```
  // ── realtime ──────────────────────────────────────────────

  useEffect(() => {
    const channel = supabase
      .channel('join-page-rooms')
      .on('postgres_changes', { event: '*', schema: 'public', table: 'rooms' }, ()
=> {
        fetchAllRooms(locationTerms);
      })
      .subscribe();
    return () => { supabase.removeChannel(channel); };
  }, [locationTerms, fetchAllRooms]);

  // ── overflow detector: given a room, find siblings under threshold ──

  const findOverflowAlternatives = useCallback((room: Room): Room[] => {
    return allOpenRooms.filter(r =>
      r.roomCode !== room.roomCode &&
      r.status === 'open' &&
      r.expiryTime > Date.now() &&
      r.totalCartValue < r.threshold &&    // hasn't hit threshold yet → needs
members
      isSameBuilding(r.building, room.building)
    ).sort((a, b) =>
      // prioritise rooms closest to threshold (most progress)
      (b.totalCartValue / b.threshold) - (a.totalCartValue / a.threshold)
    );
  }, [allOpenRooms]);

  // ── actual join (after overflow check or forced) ──────────

  const doJoin = useCallback(async (roomCode: string, roomData: Room) => {
    if (!user) { toast.error('Not authenticated yet'); return; }

    setJoining(roomCode);
    try {
      if (roomData.captainUid === user.id) {
        navigate(`/captain/${roomCode}`);
        return;
      }
      if (roomData.status !== 'open') {
        toast.error(`Room is ${roomData.status} — can't join now.`);
        return;
      }
      if (roomData.expiryTime <= Date.now()) {
        toast.error('This room has expired.');
        return;
      }

      const { data: existing } = await supabase
        .from('items').select('id').eq('room_code', roomCode).eq('member_uid',
user.id).limit(1);
```

```
      if (!existing || existing.length === 0) {
        await supabase.rpc('increment_member_count', { p_room_code: roomCode });
      }

      navigate(`/member/${roomCode}`);
    } catch (err) {
      console.error(err);
      toast.error('Something went wrong. Please try again.');
    } finally {
      setJoining(null);
    }
  }, [user, navigate]);

  // ── join handler with overflow interception ────────────────

  const handleJoin = useCallback(async (roomOrCode: Room | string, force = false) =>
{
    if (!user) { toast.error('Not authenticated yet'); return; }

    let roomData: Room | null = null;
    let roomCode: string;

    if (typeof roomOrCode === 'string') {
      roomCode = roomOrCode.trim().toUpperCase();
      if (roomCode.length !== 6) { toast.error('Room code must be 6 characters');
return; }
      const { data, error } = await supabase
        .from('rooms').select('*').eq('room_code', roomCode).single();
      if (error || !data) { toast.error('Room not found. Check the code and try
again.'); return; }
      roomData = mapRoom(data);
    } else {
      roomData = roomOrCode;
      roomCode = roomOrCode.roomCode;
    }

    // ── Overflow check ────────────────────────────────────────
    // If room already hit threshold AND there are sibling rooms that still need
    // members, intercept and show the overflow modal (unless user forced past it).
    if (!force && roomData.totalCartValue >= roomData.threshold) {
      const alts = findOverflowAlternatives(roomData);
      if (alts.length > 0) {
        setOverflowData({ original: roomData, alternatives: alts });
        return;
      }
    }

    await doJoin(roomCode, roomData);
  }, [user, findOverflowAlternatives, doJoin]);

  // ── compute which room codes are overflow targets ──────────
```

```
  const overflowTargetCodes = React.useMemo(() => {
    const set = new Set<string>();
    for (const room of allOpenRooms) {
      if (room.totalCartValue >= room.threshold) {
        const alts = allOpenRooms.filter(r =>
          r.roomCode !== room.roomCode &&
          r.status === 'open' &&
          r.expiryTime > Date.now() &&
          r.totalCartValue < r.threshold &&
          isSameBuilding(r.building, room.building)
        );
        alts.forEach(a => set.add(a.roomCode));
      }
    }
    return set;
  }, [allOpenRooms]);

  // ── render ──────────────────────────────────────────────

  const locationLabel = locationTerms.slice(0, 2).join(', ') || null;

  const renderRoomCard = (room: Room) => (
    <RoomCard
      key={room.roomCode}
      room={room}
      onJoin={handleJoin}
      joining={joining === room.roomCode}
      isOverflowTarget={overflowTargetCodes.has(room.roomCode)}
    />
  );

  return (
    <div className="min-h-screen flex flex-col">
      {/* Nav */}
      <nav className="flex items-center gap-3 px-4 sm:px-6 py-4 border-b border-
white/5 sticky top-0 bg-gray-950/80 backdrop-blur-xl z-10">
        <Link to="/" className="text-white/40 hover:text-white transition-colors p-1
rounded-lg hover:bg-white/10">
          <ArrowLeft className="w-4 h-4" />
        </Link>
        <div className="flex items-center gap-2 flex-1">
          <div className="w-7 h-7 rounded-lg bg-indigo-600 flex items-center
justify-center">
            <Zap className="w-3.5 h-3.5 text-white" />
          </div>
          <span className="font-bold text-white">Join a Room</span>
        </div>
        <button
          onClick={detectLocation}
          title="Refresh rooms"
          className="p-1.5 rounded-lg text-white/30 hover:text-white hover:bg-
```

```
white/10 transition-all"
        >
          {roomsLoading
            ? <Loader2 className="w-4 h-4 animate-spin" />
            : <RefreshCw className="w-4 h-4" />}
        </button>
      </nav>

      <div className="flex-1 max-w-lg mx-auto w-full px-4 py-6 space-y-5">

        {/* ── Location header ──────────────────────────────────── */}
        <div className="flex items-center justify-between">
          <div className="flex items-center gap-2">
            {(locStatus === 'idle' || locStatus === 'detecting') &&
nearbyRooms.length === 0 && otherRooms.length === 0 && (
              <>
                <Loader2 className="w-4 h-4 text-indigo-400 animate-spin" />
                <span className="text-white/60 text-sm font-medium">Finding open
rooms…</span>
              </>
            )}
            {locStatus === 'detecting' && (nearbyRooms.length > 0 ||
otherRooms.length > 0) && (
              <>
                <Navigation className="w-4 h-4 text-indigo-400 animate-pulse" />
                <span className="text-white/60 text-sm font-medium">Pinpointing your
location…</span>
              </>
            )}
            {locStatus === 'done' && (
              <>
                <MapPin className="w-4 h-4 text-emerald-400" />
                <span className="text-white font-semibold text-sm">
                  {locationLabel ? `Near ${locationLabel}` : 'Open rooms'}
                </span>
              </>
            )}
            {(locStatus === 'denied' || locStatus === 'error') && (
              <>
                <MapPin className="w-4 h-4 text-white/30" />
                <span className="text-white/50 text-sm">All open rooms</span>
              </>
            )}
          </div>
          {(locStatus === 'denied' || locStatus === 'error') && (
            <button
              onClick={detectLocation}
              className="flex items-center gap-1 text-indigo-400 hover:text-indigo-
300 text-xs font-semibold transition-colors"
            >
              <Navigation className="w-3 h-3" /> Enable location
            </button>
```

```
          )}
        </div>

        {/* ── Nearby rooms ──────────────────────────────────── */}
        {nearbyRooms.length > 0 && (
          <div className="space-y-3">
            <p className="text-xs font-semibold text-emerald-400/80 uppercase
tracking-wider flex items-center gap-1.5">
              <MapPin className="w-3 h-3" /> Near you
            </p>
            {nearbyRooms.map(renderRoomCard)}
          </div>
        )}

        {/* ── All other open rooms ──────────────────────────── */}
        {otherRooms.length > 0 && (
          <div className="space-y-3">
            {nearbyRooms.length > 0 && (
              <p className="text-xs font-semibold text-white/30 uppercase tracking-
wider">Other open rooms</p>
            )}
            {otherRooms.map(renderRoomCard)}
          </div>
        )}

        {/* ── Empty state ───────────────────────────────────── */}
        {!roomsLoading && nearbyRooms.length === 0 && otherRooms.length === 0
          && locStatus !== 'detecting' && locStatus !== 'idle' && (
          <div className="card p-6 text-center space-y-2 border-dashed">
            <div className="text-3xl">🏜️</div>
            <p className="text-white/60 font-medium text-sm">No open rooms right
now</p>
            <p className="text-white/30 text-xs">
              Ask your Captain for the room code, or create your own run.
            </p>
          </div>
        )}

        {/* ── Skeleton ──────────────────────────────────────── */}
        {(roomsLoading || locStatus === 'idle') && nearbyRooms.length === 0 &&
otherRooms.length === 0 && (
          <div className="space-y-3">
            {[1, 2].map(i => (
              <div key={i} className="card p-4 space-y-3 animate-pulse">
                <div className="flex justify-between">
                  <div className="space-y-2 flex-1">
                    <div className="skeleton h-4 w-24 rounded" />
                    <div className="skeleton h-5 w-48 rounded" />
                    <div className="skeleton h-3 w-32 rounded" />
                  </div>
                  <div className="skeleton h-8 w-14 rounded-lg" />
                </div>
```

```
                <div className="skeleton h-1.5 w-full rounded-full" />
              </div>
            ))}
          </div>
        )}

        {/* ── Divider ──────────────────────────────────────── */}
        <div className="flex items-center gap-3">
          <div className="flex-1 h-px bg-white/8" />
          <span className="text-white/25 text-xs font-medium uppercase tracking-
wider">or enter code manually</span>
          <div className="flex-1 h-px bg-white/8" />
        </div>

        {/* ── Manual code entry ─────────────────────────────── */}
        <div className="card p-5 space-y-4">
          <div className="flex items-center gap-2 mb-1">
            <Search className="w-3.5 h-3.5 text-white/40" />
            <p className="text-white/50 text-xs font-medium">Have a room code from
your Captain?</p>
          </div>
          <div className="flex gap-2">
            <input
              id="room-code-input"
              className="input-field flex-1 text-center text-2xl font-black
tracking-[0.35em] uppercase placeholder:text-base placeholder:font-normal
placeholder:tracking-normal"
              maxLength={6}
              placeholder="ABC123"
              value={code}
              onChange={e => setCode(e.target.value.toUpperCase())}
              onKeyDown={e => e.key === 'Enter' && code.length === 6 &&
handleJoin(code)}
            />
            <button
              id="join-room-btn"
              onClick={() => handleJoin(code)}
              disabled={!!joining || code.length < 6}
              className="btn-primary flex items-center justify-center gap-1.5 px-4
py-2.5 disabled:opacity-50 disabled:cursor-not-allowed"
            >
              {joining === code.toUpperCase()
                ? <Loader2 className="w-4 h-4 animate-spin" />
                : <><span>Join</span><ChevronRight className="w-4 h-4" /></>
              }
            </button>
          </div>
        </div>

        {/* Tip */}
        <p className="text-center text-white/25 text-xs flex items-center justify-
center gap-1.5">
```

```
          <Clock className="w-3 h-3" />
          Rooms with sibling rooms auto-suggest overflow redirection
        </p>
      </div>

      {/* ── Overflow modal ──────────────────────────────────────── */}
      {overflowData && (
        <OverflowModal
          data={overflowData}
          joining={!!joining}
          onClose={() => setOverflowData(null)}
          onStay={() => {
            const { original } = overflowData;
            setOverflowData(null);
            doJoin(original.roomCode, original);
          }}
          onSwitch={(alt) => {
            setOverflowData(null);
            doJoin(alt.roomCode, alt);
          }}
        />
      )}
    </div>
  );
};
\n```\n\n### src/pages/MemberDashboard.tsx

```\nimport React, { useEffect, useState, useCallback } from 'react';
import { useParams, useNavigate, Link } from 'react-router-dom';
import {
  ArrowLeft, Plus, ExternalLink, X, Zap, Loader2, Package, Trash2, Pencil, Check,
} from 'lucide-react';
import { supabase } from '../supabase';
import { useAuth } from '../AuthContext';
import type { Room, ItemSubmission, OOSPreference } from '../types';
import type { Platform, RoomStatus, ItemStatus } from '../types';
import { CountdownTimer } from '../components/CountdownTimer';
import { ProgressBar } from '../components/ProgressBar';
import { StatusBadge } from '../components/StatusBadge';
import { SkeletonCard, EmptyState } from '../components/Loading';
import { UPIModal } from '../components/UPIModal';
import toast from 'react-hot-toast';

// ── Save visited room to history ──────────────────────────────
function saveRoomToHistory(roomCode: string, building: string, platform: string,
role: 'captain' | 'member') {
  const key = 'droprun_room_history';
  const existing: Array<{ roomCode: string; building: string; platform: string;
role: string; visitedAt: number }> =
    JSON.parse(localStorage.getItem(key) || '[]');
  const filtered = existing.filter(r => r.roomCode !== roomCode);
  filtered.unshift({ roomCode, building, platform, role, visitedAt: Date.now() });
```
```

```
  localStorage.setItem(key, JSON.stringify(filtered.slice(0, 10)));
}

// ── DB mappers ──────────────────────────────────────────────

const mapRoom = (r: Record<string, unknown>): Room => ({
  roomCode:       r.room_code as string,
  captainUid:     r.captain_uid as string,
  captainName:    r.captain_name as string,
  captainUPI:     r.captain_upi as string,
  building:       r.building as string,
  platform:       r.platform as Platform,
  status:         r.status as RoomStatus,
  expiryTime:     r.expiry_time as number,
  threshold:      r.threshold as number,
  deliveryFee:    r.delivery_fee as number,
  totalCartValue: r.total_cart_value as number,
  memberCount:    r.member_count as number,
  createdAt:      r.created_at as number,
});

const mapItem = (r: Record<string, unknown>): ItemSubmission => ({
  id:            r.id as string,
  memberUid:     r.member_uid as string,
  memberName:    r.member_name as string,
  itemLink:      r.item_link as string,
  itemName:      r.item_name as string,
  price:         r.price as number,
  quantity:      r.quantity as number,
  subtotal:      r.subtotal as number,
  oosPreference: r.oos_preference as OOSPreference,
  status:        r.status as ItemStatus,
  submittedAt:   r.submitted_at as number,
});

// ── Add Item Form ───────────────────────────────────────────

interface AddItemFormProps {
  onClose: () => void;
  onSubmit: (data: Omit<ItemSubmission, 'id' | 'memberUid' | 'memberName' |
'submittedAt' | 'status'>) => Promise<unknown>;
  allItems: ItemSubmission[];
}

const AddItemForm: React.FC<AddItemFormProps> = ({ onClose, onSubmit, allItems }) =>
{
  const [link, setLink]       = useState('');
  const [name, setName]       = useState('');
  const [price, setPrice]     = useState('');
  const [qty, setQty]         = useState('1');
  const [oosPreference, setOos] = useState<OOSPreference>('substitute');
  const [loading, setLoading] = useState(false);
```

```
  const handleSubmit = async () => {
    if (!link.trim() || !name.trim() || !price) {
      toast.error('Fill in all required fields'); return;
    }
    const p = parseFloat(price);
    const q = parseInt(qty, 10) || 1;
    if (isNaN(p) || p <= 0) { toast.error('Enter a valid price'); return; }

    // Duplicate link check
    const duplicate = allItems.find(i => i.itemLink === link.trim());
    if (duplicate) { toast.error(`"${duplicate.itemName}" was already added by
${duplicate.memberName}`); return; }

    setLoading(true);
    try {
      await onSubmit({ itemLink: link.trim(), itemName: name.trim(), price: p,
quantity: q, subtotal: p * q, oosPreference });
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-
black/70 backdrop-blur-sm">
      <div className="card w-full max-w-md p-6 space-y-4 relative">
        <button onClick={onClose} className="absolute top-4 right-4 text-white/30
hover:text-white transition-colors">
          <X className="w-5 h-5" />
        </button>
        <h2 className="text-white font-bold text-lg">Add Item</h2>

        <div className="space-y-3">
          <input id="item-link" className="input-field" placeholder="Item URL
(Blinkit / Zepto / Instamart)" value={link} onChange={e => setLink(e.target.value)}
/>
          <input id="item-name" className="input-field" placeholder="Item name"
value={name} onChange={e => setName(e.target.value)} />
          <div className="grid grid-cols-2 gap-3">
            <div className="relative">
              <span className="absolute left-3.5 top-1/2 -translate-y-1/2 text-
white/30">₹</span>
              <input id="item-price" type="number" className="input-field pl-7"
placeholder="Price" value={price} onChange={e => setPrice(e.target.value)} min="0"
/>
            </div>
            <input id="item-qty" type="number" className="input-field"
placeholder="Qty" value={qty} onChange={e => setQty(e.target.value)} min="1" />
          </div>

          {price && qty && (
```

```
            <div className="text-right text-white/60 text-sm">
              Subtotal: <span className="text-white font-bold">₹{(parseFloat(price
|| '0') * (parseInt(qty, 10) || 1)).toFixed(0)}</span>
            </div>
          )}

          <div>
            <p className="text-white/50 text-xs mb-2">If out of stock:</p>
            <div className="grid grid-cols-2 gap-2">
              {(['substitute', 'cancel'] as OOSPreference[]).map(opt => (
                <button key={opt} onClick={() => setOos(opt)}
                  className={`py-2 rounded-xl border text-sm font-semibold
transition-all ${
                    oosPreference === opt ? 'bg-indigo-600/20 border-indigo-500
text-white' : 'border-white/10 text-white/40 hover:border-white/20'
                  }`}>
                  {opt === 'substitute' ? '↔ Substitute' : '× Cancel'}
                </button>
              ))}
            </div>
          </div>
        </div>

        <div className="grid grid-cols-2 gap-3 pt-1">
          <button onClick={onClose} className="btn-secondary py-2.5">Cancel</button>
          <button id="submit-item-btn" onClick={handleSubmit} disabled={loading}
            className="btn-primary flex items-center justify-center gap-2 py-2.5">
            {loading ? <Loader2 className="w-4 h-4 animate-spin" /> : 'Add Item'}
          </button>
        </div>
      </div>
    </div>
  );
};

// ── Member Dashboard ──────────────────────────────────────────

export const MemberDashboard: React.FC = () => {
  const { roomCode } = useParams<{ roomCode: string }>();
  const navigate = useNavigate();
  const { user, displayName } = useAuth();

  const [room, setRoom]           = useState<Room | null>(null);
  const [items, setItems]         = useState<ItemSubmission[]>([]);
  const [myPayment, setMyPayment] = useState<'pending' | 'paid'>('pending');
  const [loading, setLoading]     = useState(true);
  const [showForm, setShowForm]   = useState(false);
  const [showUPI, setShowUPI]     = useState(false);

  // ── fetch helpers ──────────────────────────────────────────

  const fetchItems = useCallback(async () => {
```

```
  if (!roomCode) return;
  const { data } = await supabase
    .from('items')
    .select('*')
    .eq('room_code', roomCode)
    .order('submitted_at', { ascending: true });
  if (data) setItems(data.map(mapItem));
}, [roomCode]);

const fetchPayment = useCallback(async () => {
  if (!roomCode || !user) return;
  const { data } = await supabase
    .from('payments')
    .select('payment_status')
    .eq('room_code', roomCode)
    .eq('member_uid', user.id)
    .single();
  if (data) setMyPayment(data.payment_status as 'pending' | 'paid');
}, [roomCode, user]);

// ── realtime ──────────────────────────────────────────

useEffect(() => {
  if (!roomCode || !user) return;
  let cancelled = false;

  const load = async () => {
    const { data: roomRow, error } = await supabase
      .from('rooms')
      .select('*')
      .eq('room_code', roomCode)
      .single();

    if (error || !roomRow) { navigate('/'); return; }
    const mapped = mapRoom(roomRow);
    if (!cancelled) { setRoom(mapped); setLoading(false); }

    // save to history
    saveRoomToHistory(roomCode, mapped.building, mapped.platform, 'member');

    await fetchItems();
    await fetchPayment();
  };

  load();

  const channel = supabase
    .channel(`member-${roomCode}-${user.id}`)
    .on('postgres_changes', {
      event: '*', schema: 'public', table: 'rooms',
      filter: `room_code=eq.${roomCode}`,
    }, (payload) => {
```

```
        if (payload.new) setRoom(mapRoom(payload.new as Record<string, unknown>));
      })
      .on('postgres_changes', {
        event: '*', schema: 'public', table: 'items',
        filter: `room_code=eq.${roomCode}`,
      }, () => fetchItems())
      .on('postgres_changes', {
        event: '*', schema: 'public', table: 'payments',
        filter: `room_code=eq.${roomCode}`,
      }, () => fetchPayment())
      .subscribe();

    return () => {
      cancelled = true;
      supabase.removeChannel(channel);
    };
  }, [roomCode, user, navigate, fetchItems, fetchPayment]);

  // ── add item ──────────────────────────────────────────────

  const handleAddItem = async (data: Omit<ItemSubmission, 'id' | 'memberUid' |
'memberName' | 'submittedAt' | 'status'>) => {
    if (!user || !roomCode) return;

    const { error } = await supabase.from('items').insert({
      room_code:      roomCode,
      member_uid:     user.id,
      member_name:    displayName || 'Member',
      item_link:      data.itemLink,
      item_name:      data.itemName,
      price:          data.price,
      quantity:       data.quantity,
      subtotal:       data.subtotal,
      oos_preference: data.oosPreference,
      status:         'pending',
      submitted_at:   Date.now(),
    });

    if (error) { toast.error('Failed to add item'); return; }

    // Atomically increment cart value
    await supabase.rpc('add_to_cart_value', {
      p_room_code: roomCode,
      p_amount:    data.subtotal,
    });

    toast.success(`"${data.itemName}" added!`);
    setShowForm(false);
  };

  // ── delete own item ───────────────────────────────────────
```

```
  const handleDeleteItem = async (item: ItemSubmission) => {
    if (!roomCode) return;
    const { error } = await supabase.from('items').delete().eq('id', item.id);
    if (error) { toast.error('Failed to delete item'); return; }
    // Decrement cart value
    await supabase.rpc('add_to_cart_value', { p_room_code: roomCode, p_amount: -
item.subtotal });
    toast.success(`"${item.itemName}" removed`);
  };

  // ── edit quantity of own item ──────────────────────────────

  const [editingItemId, setEditingItemId] = useState<string | null>(null);
  const [editQty, setEditQty] = useState('');

  const startEdit = (item: ItemSubmission) => {
    setEditingItemId(item.id!);
    setEditQty(String(item.quantity));
  };

  const saveEdit = async (item: ItemSubmission) => {
    const newQty = parseInt(editQty, 10);
    if (isNaN(newQty) || newQty < 1) { toast.error('Quantity must be at least 1');
return; }
    const newSubtotal = item.price * newQty;
    const diff = newSubtotal - item.subtotal;
    const { error } = await supabase.from('items')
      .update({ quantity: newQty, subtotal: newSubtotal })
      .eq('id', item.id);
    if (error) { toast.error('Failed to update quantity'); return; }
    if (diff !== 0) await supabase.rpc('add_to_cart_value', { p_room_code:
roomCode!, p_amount: diff });
    setEditingItemId(null);
    toast.success('Quantity updated');
  };

  // ── render ──────────────────────────────────────────────────

  if (loading) return (
    <div className="min-h-screen flex flex-col">
      <div className="p-4 border-b border-white/5"><div className="skeleton h-8 w-40
rounded-lg" /></div>
      <div className="p-4 space-y-3">{[1,2,3].map(i => <SkeletonCard key={i} />)}
</div>
    </div>
  );

  if (!room) return null;

  const myItems = items.filter(i => i.memberUid === user?.id);
  const mySubtotal = myItems.reduce((s, i) => s + i.subtotal, 0);
  const deliveryShare = room.totalCartValue > 0
```

```
    ? Math.round((mySubtotal / room.totalCartValue) * room.deliveryFee)
    : 0;
  const myTotal = mySubtotal + (room.totalCartValue >= room.threshold ? 0 :
deliveryShare);
  const isOpen = room.status === 'open';

  const platformEmoji: Record<string, string> = {
    Blinkit: '🟡', Zepto: '🟣', 'Swiggy Instamart': '🟠',
  };

  return (
    <div className="min-h-screen flex flex-col">
      {/* Nav */}
      <nav className="flex items-center justify-between px-4 sm:px-6 py-4 border-b
border-white/5 sticky top-0 bg-gray-950/80 backdrop-blur-xl z-10">
        <div className="flex items-center gap-3">
          <Link to="/" className="text-white/40 hover:text-white transition-colors
p-1 rounded-lg hover:bg-white/10">
            <ArrowLeft className="w-4 h-4" />
          </Link>
          <div>
            <div className="flex items-center gap-2">
              <span className="text-white font-bold">{room.building}</span>
              <StatusBadge status={room.status} />
            </div>
            <p className="text-white/40 text-xs">{platformEmoji[room.platform]}
{room.platform} · Captain: {room.captainName}</p>
          </div>
        </div>
        {isOpen && <CountdownTimer expiryTime={room.expiryTime} />}
      </nav>

      <div className="flex-1 max-w-2xl mx-auto w-full px-4 sm:px-6 py-6 space-y-5">

        {/* Room info */}
        <div className="card p-4 flex items-center justify-between">
          <div>
            <p className="text-white/40 text-xs">Room</p>
            <span className="text-2xl font-black text-white tracking-widest font-
mono">{room.roomCode}</span>
          </div>
          <div className="text-right">
            <p className="text-white/40 text-xs">Members</p>
            <p className="text-white font-bold">{room.memberCount}</p>
          </div>
        </div>

        {/* Progress */}
        <ProgressBar value={room.totalCartValue} threshold={room.threshold} />

        {/* Add item button */}
        {isOpen && (
```

```
          <button
            id="add-item-btn"
            onClick={() => setShowForm(true)}
            className="btn-primary w-full flex items-center justify-center gap-2 py-
3.5"
          >
            <Plus className="w-4 h-4" /> Add My Items
          </button>
        )}

        {/* My items */}
        <div className="space-y-3">
          <h2 className="text-white font-bold flex items-center gap-2">
            <Package className="w-4 h-4 text-indigo-400" />
            My Items ({myItems.length})
          </h2>

          {myItems.length === 0
            ? <EmptyState icon={isOpen ? '⏳' : '📦'} title={isOpen ? 'Add your
items!' : 'No items added'} description={isOpen ? 'Add your items before the timer
runs out.' : 'You did not add any items this run.'} />
            : myItems.map(item => (
                <div key={item.id} className={`card p-4 space-y-2 border transition-
all ${
                  item.status === 'added' ? 'border-emerald-500/30 bg-emerald-500/5'
:
                  item.status === 'oos'   ? 'border-red-500/30 bg-red-500/5' :
'border-white/8'
                }`}>
                  <div className="flex items-start justify-between gap-2">
                    <div className="flex-1 min-w-0">
                      <div className="flex items-center gap-2 flex-wrap mb-0.5">
                        {item.status === 'added'   && <span className="chip-added">✓
ADDED</span>}
                        {item.status === 'oos'     && <span className="chip-
oos">OOS</span>}
                        {item.status === 'pending' && <span className="chip-
pending">PENDING</span>}
                      </div>
                      <p className="text-white font-semibold truncate">
{item.itemName}</p>
                      <div className="flex items-center gap-2 mt-0.5">
                        {isOpen && editingItemId === item.id ? (
                          <div className="flex items-center gap-1.5">
                            <span className="text-white/40 text-sm">₹{item.price}
×</span>
                            <input
                              type="number"
                              min="1"
                              value={editQty}
                              onChange={e => setEditQty(e.target.value)}
                              className="w-14 bg-white/10 border border-indigo-
```

```
500/50 rounded-lg px-2 py-0.5 text-white text-sm text-center focus:outline-none
focus:ring-1 focus:ring-indigo-500"
                              onKeyDown={e => e.key === 'Enter' && saveEdit(item)}
                              autoFocus
                            />
                            <button onClick={() => saveEdit(item)} className="p-1
rounded-lg bg-emerald-500/20 text-emerald-400 hover:bg-emerald-500/30 transition-
colors">
                              <Check className="w-3.5 h-3.5" />
                            </button>
                            <button onClick={() => setEditingItemId(null)}
className="p-1 rounded-lg bg-white/10 text-white/40 hover:bg-white/20 transition-
colors">
                              <X className="w-3.5 h-3.5" />
                            </button>
                          </div>
                        ) : (
                          <p className="text-white/50 text-sm">
                            ₹{item.price} × {item.quantity} = <span className="text-
white font-medium">₹{item.subtotal}</span>
                          </p>
                        )}
                      </div>
                    </div>
                    {isOpen && item.status === 'pending' && editingItemId !==
item.id && (
                      <div className="flex gap-1 flex-shrink-0">
                        <button
                          id={`edit-qty-${item.id}`}
                          onClick={() => startEdit(item)}
                          title="Edit quantity"
                          className="p-1.5 rounded-lg border border-white/10 text-
white/30 hover:text-indigo-300 hover:border-indigo-500/40 hover:bg-indigo-500/10
transition-all"
                        >
                          <Pencil className="w-3.5 h-3.5" />
                        </button>
                        <button
                          id={`delete-item-${item.id}`}
                          onClick={() => handleDeleteItem(item)}
                          title="Remove item"
                          className="p-1.5 rounded-lg border border-white/10 text-
white/30 hover:text-red-400 hover:border-red-500/40 hover:bg-red-500/10 transition-
all"
                        >
                          <Trash2 className="w-3.5 h-3.5" />
                        </button>
                      </div>
                    )}
                  </div>
                  <a href={item.itemLink} target="_blank" rel="noopener noreferrer"
                    className="flex items-center gap-1.5 text-indigo-400 hover:text-
```

```
indigo-300 text-xs group transition-colors">
                    <ExternalLink className="w-3 h-3" />
                    <span className="truncate group-hover:underline">{item.itemLink}
</span>
                  </a>
                </div>
              ))
          }
        </div>

        {/* All items feed */}
        {items.filter(i => i.memberUid !== user?.id).length > 0 && (
          <div className="space-y-3">
            <h2 className="text-white/60 font-semibold text-sm flex items-center
gap-2">
              <Zap className="w-3.5 h-3.5 text-indigo-400" />
              Other Members' Items
            </h2>
            {items.filter(i => i.memberUid !== user?.id).map(item => (
              <div key={item.id} className="card p-3 flex items-center justify-
between gap-2">
                <div className="flex-1 min-w-0">
                  <p className="text-white/70 text-sm font-medium truncate">
{item.itemName}</p>
                  <p className="text-white/30 text-xs">{item.memberName} · ₹
{item.subtotal}</p>
                </div>
                {item.status === 'added'   && <span className="chip-added">✓</span>}
                {item.status === 'oos'     && <span className="chip-oos">OOS</span>}
                {item.status === 'pending' && <span className="chip-pending">…
</span>}
              </div>
            ))}
          </div>
        )}

        {/* Payment section */}
        {(room.status === 'locked' || room.status === 'ordering' || room.status ===
'delivered') && mySubtotal > 0 && (
          <div className={`card p-5 space-y-4 border ${myPayment === 'paid' ?
'border-emerald-500/20 bg-emerald-500/5' : 'border-amber-500/20 bg-amber-500/5'}`}>
            <div className="flex items-center justify-between">
              <h3 className="text-white font-bold">Your Total</h3>
              {myPayment === 'paid'
                ? <span className="chip-paid">✓ PAID</span>
                : <span className="chip-pending">UNPAID</span>
              }
            </div>

            <div className="space-y-1.5 text-sm">
              <div class="flex justify-between text-white/60">
                <span>Items subtotal</span>
```

```
                <span>₹{mySubtotal}</span>
              </div>
              <div className="flex justify-between text-white/60">
                <span>Delivery share</span>
                <span>{room.totalCartValue >= room.threshold ? <span
className="text-emerald-400">Free 🎉</span> : `₹${deliveryShare}`}</span>
              </div>
              <div className="flex justify-between text-white font-bold text-base
border-t border-white/10 pt-2 mt-2">
                <span>Total to pay {room.captainName}</span>
                <span>₹{myTotal}</span>
              </div>
            </div>

            {myPayment !== 'paid' && (
              <button
                id="pay-captain-btn"
                onClick={() => setShowUPI(true)}
                className="btn-primary w-full flex items-center justify-center gap-2
py-3"
              >
                Pay ₹{myTotal} via UPI
              </button>
            )}
          </div>
        )}

      </div>

      {showForm && (
        <AddItemForm
          onClose={() => setShowForm(false)}
          onSubmit={handleAddItem}
          allItems={items}
        />
      )}

      {showUPI && room && user && (
        <UPIModal
          amount={myTotal}
          captainUPI={room.captainUPI!}
          captainName={room.captainName}
          roomCode={room.roomCode}
          memberUid={user.id}
          memberName={displayName || 'Member'}
          onClose={() => setShowUPI(false)}
          onPaid={() => setMyPayment('paid')}
        />
      )}
    </div>
  );
};
```

\n```\n\n### src/pages/PaymentSplitsPage.tsx

```\nimport React, { useEffect, useState, useCallback } from 'react';
import { useParams, Link } from 'react-router-dom';
import { ArrowLeft, Zap, Check, Users } from 'lucide-react';
import { supabase } from '../supabase';
import { useAuth } from '../AuthContext';
import type { Room, ItemSubmission, MemberPayment } from '../types';
import type { Platform, RoomStatus, OOSPreference, ItemStatus } from '../types';
import { UPIModal } from '../components/UPIModal';
import { SkeletonCard } from '../components/Loading';
import toast from 'react-hot-toast';
import { calcMemberDeliveryShare } from '../utils';

// ── DB mappers ──────────────────────────────────────────────

const mapRoom = (r: Record<string, unknown>): Room => ({
  roomCode:       r.room_code as string,
  captainUid:     r.captain_uid as string,
  captainName:    r.captain_name as string,
  captainUPI:     r.captain_upi as string,
  building:       r.building as string,
  platform:       r.platform as Platform,
  status:         r.status as RoomStatus,
  expiryTime:     r.expiry_time as number,
  threshold:      r.threshold as number,
  deliveryFee:    r.delivery_fee as number,
  totalCartValue: r.total_cart_value as number,
  memberCount:    r.member_count as number,
  createdAt:      r.created_at as number,
});

const mapItem = (r: Record<string, unknown>): ItemSubmission => ({
  id:            r.id as string,
  memberUid:     r.member_uid as string,
  memberName:    r.member_name as string,
  itemLink:      r.item_link as string,
  itemName:      r.item_name as string,
  price:         r.price as number,
  quantity:      r.quantity as number,
  subtotal:      r.subtotal as number,
  oosPreference: r.oos_preference as OOSPreference,
  status:        r.status as ItemStatus,
  submittedAt:   r.submitted_at as number,
});

// ── Component ───────────────────────────────────────────────

export const PaymentSplitsPage: React.FC = () => {
  const { roomCode } = useParams<{ roomCode: string }>();
  const { user, displayName } = useAuth();
```

```
  const [room, setRoom]             = useState<Room | null>(null);
  const [items, setItems]           = useState<ItemSubmission[]>([]);
  const [payments, setPayments]     = useState<Record<string, string>>({});
  const [loading, setLoading]       = useState(true);
  const [selectedMember, setSelected] = useState<MemberPayment | null>(null);

  // ── fetch helpers ──────────────────────────────────────────

  const fetchAll = useCallback(async () => {
    if (!roomCode) return;

    const [roomRes, itemsRes, paymentsRes] = await Promise.all([
      supabase.from('rooms').select('*').eq('room_code', roomCode).single(),
      supabase.from('items').select('*').eq('room_code', roomCode),
      supabase.from('payments').select('*').eq('room_code', roomCode),
    ]);

    if (roomRes.data)     setRoom(mapRoom(roomRes.data));
    if (itemsRes.data)    setItems(itemsRes.data.map(mapItem));
    if (paymentsRes.data) {
      const map: Record<string, string> = {};
      for (const p of paymentsRes.data) map[p.member_uid] = p.payment_status;
      setPayments(map);
    }

    setLoading(false);
  }, [roomCode]);

  // ── realtime ───────────────────────────────────────────────

  useEffect(() => {
    if (!roomCode) return;

    fetchAll();

    const channel = supabase
      .channel(`splits-${roomCode}`)
      .on('postgres_changes', { event: '*', schema: 'public', table: 'rooms',
filter: `room_code=eq.${roomCode}` }, () => fetchAll())
      .on('postgres_changes', { event: '*', schema: 'public', table: 'items',
filter: `room_code=eq.${roomCode}` }, () => fetchAll())
      .on('postgres_changes', { event: '*', schema: 'public', table: 'payments',
filter: `room_code=eq.${roomCode}` }, () => fetchAll())
      .subscribe();

    return () => { supabase.removeChannel(channel); };
  }, [roomCode, fetchAll]);

  // ── compute splits ─────────────────────────────────────────

  const computeSplits = (): MemberPayment[] => {
    if (!room) return [];
```

```
    const byMember: Record<string, { name: string; items: ItemSubmission[] }> = {};
    for (const item of items) {
      if (!byMember[item.memberUid]) byMember[item.memberUid] = { name:
item.memberName, items: [] };
      byMember[item.memberUid].items.push(item);
    }

    return Object.entries(byMember).map(([uid, { name, items: memberItems }]) => {
      const itemCount   = memberItems.reduce((s, i) => s + i.quantity, 0);
      const subtotal    = memberItems.reduce((s, i) => s + i.subtotal, 0);
      const freeDelivery = room.totalCartValue >= room.threshold;
      const deliveryShare = freeDelivery ? 0 : calcMemberDeliveryShare(subtotal,
room.totalCartValue, room.deliveryFee);
      const total        = subtotal + deliveryShare;
      const paymentStatus = (payments[uid] === 'paid' ? 'paid' : 'pending') as
'paid' | 'pending';

      return { memberUid: uid, memberName: name, itemCount, subtotal, deliveryShare,
total, paymentStatus };
    });
  };

  const togglePayment = async (mp: MemberPayment) => {
    if (!roomCode) return;
    const newStatus = mp.paymentStatus === 'paid' ? 'pending' : 'paid';
    const { error } = await supabase.from('payments').upsert(
      { room_code: roomCode, member_uid: mp.memberUid, member_name: mp.memberName,
payment_status: newStatus },
      { onConflict: 'room_code,member_uid' }
    );
    if (error) { toast.error('Failed to update payment'); return; }
    toast.success(`${mp.memberName} marked as ${newStatus}`);
  };

  // ── render ──────────────────────────────────────────────

  if (loading) return (
    <div className="min-h-screen flex flex-col">
      <div className="p-4 border-b border-white/5"><div className="skeleton h-8 w-40
rounded-lg" /></div>
      <div className="p-6 space-y-3">{[1,2,3].map(i => <SkeletonCard key={i} />)}
</div>
    </div>
  );

  if (!room) return <p className="text-white text-center p-10">Room not found</p>;

  const splits = computeSplits();
  const allPaid = splits.length > 0 && splits.every(s => s.paymentStatus ===
'paid');
  const totalCollected = splits.filter(s => s.paymentStatus === 'paid').reduce((a,
```

```
s) => a + s.total, 0);
  const totalOwed      = splits.reduce((a, s) => a + s.total, 0);
  const isCaptain      = user?.id === room.captainUid;
  const freeDelivery   = room.totalCartValue >= room.threshold;

  return (
    <div className="min-h-screen flex flex-col">
      <nav className="flex items-center gap-3 px-4 sm:px-6 py-4 border-b border-
white/5 sticky top-0 bg-gray-950/80 backdrop-blur-xl z-10">
        <Link to={isCaptain ? `/captain/${roomCode}` : `/member/${roomCode}`}
          className="text-white/40 hover:text-white transition-colors p-1 rounded-lg
hover:bg-white/10">
          <ArrowLeft className="w-4 h-4" />
        </Link>
        <div className="flex items-center gap-2">
          <div className="w-7 h-7 rounded-lg bg-indigo-600 flex items-center
justify-center">
            <Users className="w-3.5 h-3.5 text-white" />
          </div>
          <span className="font-bold text-white">Payment Splits</span>
        </div>
      </nav>

      <div className="flex-1 max-w-2xl mx-auto w-full px-4 sm:px-6 py-6 space-y-5">

        {/* Summary header */}
        <div className="card p-5 space-y-3">
          <div className="flex items-center justify-between">
            <div>
              <p className="text-white/40 text-xs">Room</p>
              <p className="text-white font-black tracking-wider text-xl">
{room.roomCode}</p>
            </div>
            <div className="text-right">
              <p className="text-white/40 text-xs">Status</p>
              <p className={`font-bold text-sm ${allPaid ? 'text-emerald-400' :
'text-amber-400'}`}>
                {allPaid ? '✓ All Settled' : `${splits.filter(s => s.paymentStatus
=== 'pending').length} Pending`}
              </p>
            </div>
          </div>

          <div className="grid grid-cols-3 gap-3 pt-1 border-t border-white/8">
            <div className="text-center">
              <p className="text-white font-black text-xl">₹{room.totalCartValue}
</p>
              <p className="text-white/40 text-xs">Cart Value</p>
            </div>
            <div className="text-center">
              <p className={`font-black text-xl ${freeDelivery ? 'text-emerald-400'
: 'text-white'}`}>
```

```
                {freeDelivery ? <span>Free 🎉</span> : `₹${room.deliveryFee}`}
              </p>
              <p className="text-white/40 text-xs">Delivery</p>
            </div>
            <div className="text-center">
              <p className="text-white font-black text-xl">
                ₹{totalCollected}<span className="text-white/30 font-normal text-
sm">/{totalOwed}</span>
              </p>
              <p className="text-white/40 text-xs">Collected</p>
            </div>
          </div>
        </div>

        {/* Splits table */}
        <div className="card overflow-hidden">
          <div className="grid grid-cols-5 gap-2 px-4 py-3 border-b border-white/8
text-white/40 text-xs font-semibold uppercase tracking-wider">
            <span className="col-span-2">Member</span>
            <span className="text-right">Items</span>
            <span className="text-right">Amount</span>
            <span className="text-right">Status</span>
          </div>

          {splits.length === 0
            ? <div className="px-4 py-8 text-center text-white/30 text-sm">No items
submitted yet</div>
            : splits.map(mp => (
                <div key={mp.memberUid}
                  className={`grid grid-cols-5 gap-2 px-4 py-3.5 border-b border-
white/5 last:border-0 items-center hover:bg-white/3 transition-colors ${mp.memberUid
=== user?.id ? 'bg-indigo-500/5' : ''}`}>
                  <div className="col-span-2 flex items-center gap-2.5">
                    <div className={`w-7 h-7 rounded-full flex items-center justify-
center text-xs font-bold flex-shrink-0 ${
                      mp.paymentStatus === 'paid' ? 'bg-emerald-500/20 text-emerald-
400' : 'bg-indigo-500/20 text-indigo-300'}`}>
                      {mp.memberName.charAt(0).toUpperCase()}
                    </div>
                    <div className="min-w-0">
                      <p className="text-white text-sm font-medium truncate">
{mp.memberName}</p>
                      {mp.memberUid === user?.id && <p className="text-indigo-400
text-xs">You</p>}
                    </div>
                  </div>
                  <div className="text-right text-white/60 text-sm">{mp.itemCount}
</div>
                  <div className="text-right">
                    <p className="text-white font-bold text-sm">₹{mp.total}</p>
                    {!freeDelivery && <p className="text-white/30 text-xs">+₹
{mp.deliveryShare} del.</p>}
```

```
                  </div>
                  <div className="flex justify-end">
                    {isCaptain ? (
                      <button
                        onClick={() => togglePayment(mp)}
                        className={`flex items-center gap-1 px-2.5 py-1.5 rounded-lg
border text-xs font-semibold transition-all ${
                          mp.paymentStatus === 'paid'
                            ? 'bg-emerald-500/20 border-emerald-500/30 text-emerald-
400 hover:bg-red-500/10 hover:border-red-500/30 hover:text-red-400'
                            : 'border-white/10 text-white/40 hover:bg-emerald-500/10
hover:border-emerald-500/30 hover:text-emerald-400'
                        }`}
                      >
                        {mp.paymentStatus === 'paid' ? <><Check className="w-3 h-3"
/> PAID</> : 'Mark paid'}
                      </button>
                    ) : mp.memberUid === user?.id ? (
                      mp.paymentStatus === 'paid'
                        ? <span className="chip-paid flex items-center gap-1"><Check
className="w-3 h-3" /> PAID</span>
                        : <button id="pay-from-splits-btn" onClick={() =>
setSelected(mp)}
                            className="btn-primary text-xs py-1.5 px-3">
                            Pay ₹{mp.total}
                          </button>
                    ) : (
                      <span className={mp.paymentStatus === 'paid' ? 'chip-paid' :
'chip-pending'}>
                        {mp.paymentStatus === 'paid' ? '✓ PAID' : 'UNPAID'}
                      </span>
                    )}
                  </div>
                </div>
              ))
          }

          {/* Total row */}
          <div className="grid grid-cols-5 gap-2 px-4 py-3.5 border-t border-
white/10 bg-white/3 items-center">
            <span className="col-span-2 text-white/60 text-sm font-semibold
uppercase tracking-wider">Total</span>
            <span className="text-right text-white/60 text-sm">{splits.reduce((a,s)
=> a+s.itemCount, 0)}</span>
            <span className="text-right text-white font-black">₹{totalOwed}</span>
            <span className="text-right">
              {allPaid
                ? <span className="chip-paid">All ✓</span>
                : <span className="text-white/40 text-xs">₹{totalCollected}
in</span>
              }
            </span>
```

```
          </div>
        </div>

        {/* UPI QR for captain */}
        {isCaptain && (
          <div className="card p-4 space-y-2 border border-indigo-500/20 bg-indigo-
500/5">
            <div className="flex items-center gap-2">
              <Zap className="w-4 h-4 text-indigo-400" />
              <p className="text-white font-semibold text-sm">Your UPI: <span
className="font-mono text-indigo-300">{room.captainUPI}</span></p>
            </div>
            <p className="text-white/40 text-xs">Share this with members so they can
pay you directly</p>
          </div>
        )}

        {allPaid && (
          <div className="card p-6 text-center space-y-2 border border-emerald-
500/20 bg-emerald-500/5">
            <div className="text-4xl">🎉</div>
            <h3 className="text-white font-bold">All Settled!</h3>
            <p className="text-white/50 text-sm">Everyone has paid. Great run!</p>
          </div>
        )}
      </div>

      {selectedMember && room && user && (
        <UPIModal
          amount={selectedMember.total}
          captainUPI={room.captainUPI!}
          captainName={room.captainName}
          roomCode={room.roomCode}
          memberUid={user.id}
          memberName={displayName || 'Member'}
          onClose={() => setSelected(null)}
          onPaid={() => {
            setPayments(prev => ({ ...prev, [user.id]: 'paid' }));
            setSelected(null);
          }}
        />
      )}
    </div>
  );
};
\n```\n\n### src/supabase.ts

```\nimport { createClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL as string;
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY as string;
```
```

```
export const supabase = createClient(supabaseUrl, supabaseAnonKey);
\n```\n\n### src/types.ts

```\n// ============================================================
// Shared TypeScript types for DropRun
// ==

export type RoomStatus = 'open' | 'locked' | 'ordering' | 'delivered' | 'dissolved';
export type ItemStatus = 'pending' | 'added' | 'oos';
export type OOSPreference = 'substitute' | 'cancel';
export type Platform = 'Blinkit' | 'Zepto' | 'Swiggy Instamart';
export type PaymentStatus = 'pending' | 'paid';

export interface Room {
  roomCode: string;
  captainUid: string;
  captainName: string;
  captainUPI?: string;
  building: string;
  platform: Platform;
  status: RoomStatus;
  expiryTime: number; // unix ms timestamp
  threshold: number;
  deliveryFee: number;
  totalCartValue: number;
  memberCount: number;
  createdAt: number;
}

export interface ItemSubmission {
  id?: string;
  memberUid: string;
  memberName: string;
  itemLink: string;
  itemName: string;
  price: number;
  quantity: number;
  subtotal: number;
  oosPreference: OOSPreference;
  status: ItemStatus;
  submittedAt: number;
  paymentStatus?: PaymentStatus;
}

export interface MemberPayment {
  memberUid: string;
  memberName: string;
  subtotal: number;
  deliveryShare: number;
  total: number;
  paymentStatus: PaymentStatus;
  itemCount: number;
```
```

```
}

export interface DemoRoom extends Room {
  items: ItemSubmission[];
  payments: MemberPayment[];
}
\n```\n\n### src/utils.ts

```\nimport type { Room, ItemSubmission, MemberPayment } from './types';

// ==
// Room Code Generation
// ==
export function generateRoomCode(): string {
  const chars = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';
  let code = '';
  for (let i = 0; i < 6; i++) {
    code += chars[Math.floor(Math.random() * chars.length)];
  }
  return code;
}

// ==
// Countdown helpers
// ==
export function formatCountdown(ms: number): string {
  if (ms <= 0) return '00:00';
  const totalSec = Math.floor(ms / 1000);
  const mins = Math.floor(totalSec / 60);
  const secs = totalSec % 60;
  return `${String(mins).padStart(2, '0')}:${String(secs).padStart(2, '0')}`;
}

export function getCountdownClass(ms: number): string {
  if (ms <= 60_000) return 'countdown-red';
  if (ms <= 180_000) return 'countdown-amber';
  return '';
}

// ==
// UPI deep-link generator
// ==
export function generateUPILink(params: {
  upiId: string;
  name: string;
  amount: number;
  roomCode: string;
}): string {
  const note = encodeURIComponent(`DropRun Room${params.roomCode}`);
  const pn = encodeURIComponent(params.name);
  return `upi://pay?
pa=${params.upiId}&pn=${pn}&am=${params.amount.toFixed(2)}&cu=INR&tn=${note}`;
```
```

```
}

// ============================================================
// Delivery fee calculation
// ============================================================
export function calcDeliveryFee(totalCartValue: number, threshold: number): number {
  return totalCartValue >= threshold ? 0 : 30;
}

export function calcMemberDeliveryShare(
  memberSubtotal: number,
  totalCartValue: number,
  deliveryFee: number
): number {
  if (totalCartValue === 0 || deliveryFee === 0) return 0;
  return Math.round((memberSubtotal / totalCartValue) * deliveryFee);
}

// ============================================================
// DEMO DATA (pre-seeded completed run)
// ============================================================
const NOW = Date.now();

export const DEMO_ROOM: Room = {
  roomCode: 'DEMO01',
  captainUid: 'captain-demo',
  captainName: 'Arjun S.',
  captainUPI: 'arjun@upi',
  building: 'Hostel Block C — IIT Delhi',
  platform: 'Blinkit',
  status: 'delivered',
  expiryTime: NOW - 3_600_000,
  threshold: 200,
  deliveryFee: 0,
  totalCartValue: 680,
  memberCount: 4,
  createdAt: NOW - 5_400_000,
};

export const DEMO_ITEMS: ItemSubmission[] = [
  {
    id: 'd1',
    memberUid: 'uid-rahul',
    memberName: 'Rahul',
    itemLink: 'https://blinkit.com/prn/lays-magic-masala-26g/prid/397662',
    itemName: 'Lays Magic Masala 26g',
    price: 20,
    quantity: 3,
    subtotal: 60,
    oosPreference: 'substitute',
    status: 'added',
    submittedAt: NOW - 5_000_000,
```

```
  },
  {
    id: 'd2',
    memberUid: 'uid-rahul',
    memberName: 'Rahul',
    itemLink: 'https://blinkit.com/prn/red-bull/prid/123456',
    itemName: 'Red Bull 250ml',
    price: 120,
    quantity: 1,
    subtotal: 120,
    oosPreference: 'cancel',
    status: 'added',
    submittedAt: NOW - 4_900_000,
  },
  {
    id: 'd3',
    memberUid: 'uid-priya',
    memberName: 'Priya',
    itemLink: 'https://blinkit.com/prn/amul-milk/prid/234567',
    itemName: 'Amul Toned Milk 500ml',
    price: 30,
    quantity: 2,
    subtotal: 60,
    oosPreference: 'substitute',
    status: 'added',
    submittedAt: NOW - 4_800_000,
  },
  {
    id: 'd4',
    memberUid: 'uid-karan',
    memberName: 'Karan',
    itemLink: 'https://blinkit.com/prn/maggi-noodles/prid/345678',
    itemName: 'Maggi 2-Minute Noodles 70g × 4',
    price: 80,
    quantity: 2,
    subtotal: 160,
    oosPreference: 'substitute',
    status: 'added',
    submittedAt: NOW - 4_700_000,
  },
  {
    id: 'd5',
    memberUid: 'uid-sneha',
    memberName: 'Sneha',
    itemLink: 'https://blinkit.com/prn/amul-butter/prid/456789',
    itemName: 'Amul Butter 100g',
    price: 55,
    quantity: 2,
    subtotal: 110,
    oosPreference: 'cancel',
    status: 'added',
    submittedAt: NOW - 4_600_000,
```

```
  },
  {
    id: 'd6',
    memberUid: 'uid-sneha',
    memberName: 'Sneha',
    itemLink: 'https://blinkit.com/prn/bread/prid/567890',
    itemName: 'Modern Bread White 300g',
    price: 40,
    quantity: 1,
    subtotal: 40,
    oosPreference: 'substitute',
    status: 'oos',
    submittedAt: NOW - 4_500_000,
  },
];

export const DEMO_PAYMENTS: MemberPayment[] = [
  { memberUid: 'uid-rahul',  memberName: 'Rahul', subtotal: 180, deliveryShare: 0,
total: 180, paymentStatus: 'paid', itemCount: 2 },
  { memberUid: 'uid-priya',  memberName: 'Priya', subtotal: 60,  deliveryShare: 0,
total: 60,  paymentStatus: 'paid', itemCount: 1 },
  { memberUid: 'uid-karan',  memberName: 'Karan', subtotal: 160, deliveryShare: 0,
total: 160, paymentStatus: 'paid', itemCount: 1 },
  { memberUid: 'uid-sneha',  memberName: 'Sneha', subtotal: 150, deliveryShare: 0,
total: 150, paymentStatus: 'paid', itemCount: 2 },
];
\n```\n\n### vite.config.ts

```\nimport { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
  optimizeDeps: {
    include: ['tslib', '@supabase/supabase-js'],
  },
})
\n```\n\n
```
```